Teste 6 da PoC (v2): Baseline centralizado × STEP-GAN Federado com SMPC (modelo final do Teste 4 v5)

Questão de pesquisa: o STEP-GAN treinado de forma federada com agregação segura (SMPC) alcança desempenho de detecção de fraude **não-inferior** ao mesmo modelo treinado de forma **centralizada** (todos os dados num único servidor, sem privacidade)?

Modelo: as seções 0 a 4 deste notebook são **idênticas** às do Teste 4 (v5): STEP-GAN com geradores condicionados por limiares e Discriminador graduado, SMPC em Z_p (p = 2^61−1), mesmos dados e hiperparâmetros. Só a seção 5 (o experimento) é específica deste teste.

Configuração	Papel no experimento
Centralizado	Baseline: a união dos dados dos 3 clientes num único modelo
Federado + SMPC	Modelo final (Teste 4 v5)
Local isolado	Limite inferior: cada banco treina sozinho (média dos 3 clientes)

Comparação justa (mesmo procedimento para todos):

- mesmos pesos iniciais (D e os K geradores) por semente;
- mesmo orçamento: `NUM_RODADAS` blocos de `EPOCAS_LOCAIS` épocas (35 no total). No centralizado, cada bloco corresponde a uma rodada federada, com o otimizador reiniciado como no FL;
- **mesma seleção de modelo:** o checkpoint com melhor PR-AUC na validação, avaliado ao fim de cada bloco/rodada;
- limiar escolhido só na validação e métricas no teste.

Estatística: `SEMENTES` execuções pareadas; Δ = Federado+SMPC − Centralizado por semente (média, IC 95% t) e IC 95% por bootstrap do Δ médio no teste. É **não-inferior** se o limite inferior do IC do Δ da `METRICA_PRINCIPAL` for maior que −`MARGEM_NAO_INFERIORIDADE`. O resultado é reportado para ROC-AUC e PR-AUC.

```
# -*- coding: utf-8 -*-
"""
Teste 6 da PoC (v2): Baseline centralizado do modelo final (seções 0-4 idênticas ao Teste 4 v5)
- STEP-GAN: K geradores condicionados por limiares (degraus da saída do Discriminador)
- Aprendizado Federado (3 silos) com agregação SMPC modular (p = 2^61 - 1)
- Avaliação com várias sementes (média ± desvio-padrão)
"""

import os
import copy
import time
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             precision_recall_curve)

try:
    import torch
    import torch.nn as nn
    import torch.optim as optim
    from torch.utils.data import DataLoader, TensorDataset
except ImportError:
    raise ImportError("PyTorch não está instalado. Execute: pip install torch")

# ======================================================================
# --- 0. CONFIGURAÇÃO ---
# ======================================================================
SEMENTES = [42, 43, 44, 45, 46]    # 5 a 10 sementes (comentário 1)
NUM_CLIENTES = 3
NUM_RODADAS = 5                     # Rodadas de comunicação do FL
EPOCAS_LOCAIS = 7                   # 5 x 7 = 35 épocas, mesmo total da versão anterior

# Hiperparâmetros do Teste 4
HP_BATCH_SIZE = 512
HP_LR_G = 0.0002
HP_LR_D = 0.0001
HP_LATENT_DIM = 100
HP_WEIGHT_DECAY = 1e-4
```

```
PESO_FRAUDE = 15.0                  # Loss ponderada para fraudes reais
ALVO_NORMAL = 0.9                   # Label smoothing: normal real
ALVO_ANOMALIA = 0.1                 # Fraude real e amostras geradas
PRECISAO_MINIMA = 0.70              # Limiar: maior recall com precisão >= 70% (na validação)
RAZAO_NORMAIS_POR_FRAUDE = 200      # Amostragem: 200 normais para cada fraude

# STEP-GAN: um gerador por degrau da saída do Discriminador (comentário 3)
# Degraus entre ALVO_ANOMALIA (0.1) e ALVO_NORMAL (0.9): pseudo-anomalias de "longe" a "perto" da fronteira
DEGRAUS = [(0.10, 0.35), (0.35, 0.60), (0.60, 0.85)]
DIM_CONDICAO = 16
# Discriminador graduado: a amostra de G_k(z, τ) recebe alvo (τ - MARGEM_GRADUADA) no Discriminador, em vez de 0.1.
# Assim D e G_k concordam sobre o degrau e o condicionamento passa a ser efetivo. Com todas as amostras geradas
# rotuladas como 0.1 (versão anterior), o condicionamento só funcionava para τ <= ~0.5 ou o desempenho caía.
# Ablação (Credit Card, semente 42): margem 0.0 -> PR-AUC 0.855 e correlação τ×D ~0.97; margem 0.15 -> PR-AUC 0.668.
DISCRIMINADOR_GRADUADO = True
MARGEM_GRADUADA = 0.0

# SMPC (comentário 2)
SMPC_MODULO = 2**61 - 1             # Primo de Mersenne; partes em [0, p) e somas de 2 partes < 2^62 cabem em int64
SMPC_BITS_PRECISAO = 24             # Resolução do ponto fixo: 2^-24 ≈ 6e-8
SMPC_ESCALA = 2 ** SMPC_BITS_PRECISAO
SMPC_LIMITE_SOMA = (SMPC_MODULO // 2) / SMPC_ESCALA   # |Σ| representável ≈ 6.9e10

DATASETS = ['paysim', 'creditcard']
CAMINHOS = {
    'creditcard': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv',
        'base_de_dados/creditcard.zip',        # Executando de dentro de PoC/
        'PoC/base_de_dados/creditcard.zip',    # Executando da raiz do repositório
    ],
    'paysim': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv',
    ],
}


def seed_everything(seed):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


sns.set_style("whitegrid")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[SETUP] Dispositivo PyTorch: {device}")


def sincronizar():
    """Garante que o cronômetro só pare depois que a GPU terminar (CUDA é assíncrono)."""
    if device.type == 'cuda':
        torch.cuda.synchronize()


try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Fora do Google Colab: usando apenas caminhos locais.")
```

```
[SETUP] Dispositivo PyTorch: cuda
Mounted at /content/drive
```

```
# ======================================================================
# --- 1. PROTOCOLO SMPC: COMPARTILHAMENTO ADITIVO EM Z_p (comentário 2) ---
# ======================================================================

def codificar_ponto_fixo(v):
    """
    float64 -> Z_p. Arredonda para o inteiro mais próximo de v * 2^24 e representa negativos como p - |x|.
    Ex.: -1.5 -> round(-1.5 * 2^24) = -25165824 -> p - 25165824.
    """
    return torch.remainder(torch.round(v * SMPC_ESCALA).to(torch.int64), SMPC_MODULO)


def decodificar_ponto_fixo(x):
    """Z_p -> float64. Valores acima de p/2 são interpretados como negativos."""
    x = torch.where(x > SMPC_MODULO // 2, x - SMPC_MODULO, x)
    return x.to(torch.float64) / SMPC_ESCALA
```

```
def mascaras_csprng(forma):
    """Inteiros uniformes em [0, p) gerados com os.urandom (CSPRNG do sistema operacional)."""
    n = int(np.prod(forma))
    brutos = np.frombuffer(os.urandom(8 * n), dtype=np.uint64) & np.uint64(SMPC_MODULO)  # 61 bits
    brutos = brutos % np.uint64(SMPC_MODULO)                                             # mapeia p -> 0
    return torch.from_numpy(brutos.astype(np.int64)).view(forma)


def soma_modular(tensores):
    """Soma reduzindo mod p a cada passo: cada parcela < 2^61, então a soma de duas < 2^62 nunca estoura o int64."""
    total = torch.zeros_like(tensores[0])
    for t in tensores:
        total = torch.remainder(total + t, SMPC_MODULO)
    return total


def gerar_partes(x_int, n):
    """Divide x (já em Z_p) em n partes aleatórias com Σ partes ≡ x (mod p). Cada parte isolada é uniforme em Z_p."""
    mascaras = [mascaras_csprng(x_int.shape) for _ in range(n - 1)]
    ultima = x_int
    for m in mascaras:
        ultima = torch.remainder(ultima - m, SMPC_MODULO)  # diferença em (-p, p): sem estouro
    return mascaras + [ultima]


def verificar_faixa(v, n):
    """
    Checagem de estouro feita por cada cliente antes de compartilhar: se todo |v_k| <= LIMITE / n,
    então |Σ v_k| < p / 2 e a decodificação de negativos continua correta.
    """
    maximo = v.abs().max().item()
    if maximo > SMPC_LIMITE_SOMA / n:
        raise ValueError(f"Estouro do módulo SMPC: |valor| = {maximo:.3e} > {SMPC_LIMITE_SOMA / n:.3e}. "
                         f"Aumente SMPC_MODULO ou reduza SMPC_BITS_PRECISAO.")


def achatar(estado):
    """Concatena todos os tensores de ponto flutuante do estado (pesos e buffers do BatchNorm) em float64."""
    return torch.cat([t.detach().cpu().to(torch.float64).flatten()
                      for t in estado.values() if t.is_floating_point()])


def desachatar(vetor, estado_ref, estados_clientes):
    novo, pos = {}, 0
    for chave, t in estado_ref.items():
        if t.is_floating_point():
            novo[chave] = vetor[pos:pos + t.numel()].view(t.shape).to(t.dtype)
            pos += t.numel()
        else:
            # num_batches_tracked do BatchNorm: contador inteiro sem informação dos dados (não é usado com momentum)
            novo[chave] = torch.stack([e[chave] for e in estados_clientes]).max()
    return novo


def agregar_smpc(estados_clientes, pesos):
    """
    FedAvg ponderado via SMPC com n nós (os próprios clientes):
      1. Cliente k: pondera (w_k * n_k/N), verifica a faixa, codifica em Z_p e gera n partes; a parte j vai para o nó j.
      2. Nó j: soma (mod p) apenas as partes que recebeu, que são individualmente uniformes (não revelam nada).
      3. Servidor: soma (mod p) as n somas parciais e decodifica. Ele só conhece o agregado Σ_k w_k n_k/N.
    """
    n = len(estados_clientes)
    t0 = time.perf_counter()
    caixas = [[] for _ in range(n)]                      # caixas[j]: partes recebidas pelo nó j
    for estado, peso in zip(estados_clientes, pesos):
        v = achatar(estado) * peso
        verificar_faixa(v, n)
        for j, parte in enumerate(gerar_partes(codificar_ponto_fixo(v), n)):
            caixas[j].append(parte)
    somas_parciais = [soma_modular(caixas[j]) for j in range(n)]
    agregado = decodificar_ponto_fixo(soma_modular(somas_parciais))
    t_smpc = time.perf_counter() - t0

    referencia = sum(achatar(e) * p for e, p in zip(estados_clientes, pesos))   # FedAvg em claro
    erro_max = (agregado - referencia).abs().max().item()
    bytes_cliente = n * agregado.numel() * 8           # (n-1) partes aos outros nós + 1 soma parcial
    return desachatar(agregado, estados_clientes[0], estados_clientes), dict(
        t_smpc=t_smpc, erro_max=erro_max, bytes_cliente=bytes_cliente)


def autoteste_smpc():
```

```
    print("--- Autoteste do SMPC ---")
    n, pesos = 3, [1 / 3] * 3
    valores = [torch.tensor([-1.5, 30.0, -30.0, 1595.5, -2.0e7, 7e-8, 0.123456789], dtype=torch.float64) * f
               for f in (1.0, 0.5, -0.25)]
    esperado = sum(v * p for v, p in zip(valores, pesos))
    caixas = [[] for _ in range(n)]
    for v, p in zip(valores, pesos):
        for j, parte in enumerate(gerar_partes(codificar_ponto_fixo(v * p), n)):
            caixas[j].append(parte)
    obtido = decodificar_ponto_fixo(soma_modular([soma_modular(c) for c in caixas]))
    erro = (obtido - esperado).abs().max().item()
    tolerancia = n * 0.5 / SMPC_ESCALA
    assert erro <= tolerancia, f"Erro de reconstrução {erro} > {tolerancia}"
    print(f"  Reconstrução com negativos e valores grandes: erro máx. {erro:.2e} (tolerância {tolerancia:.2e}) OK")

    parte_isolada = decodificar_ponto_fixo(caixas[0][0])
    print(f"  Uma parte isolada decodificada parece ruído: {parte_isolada[:3].tolist()}")

    try:
        verificar_faixa(torch.tensor([SMPC_LIMITE_SOMA], dtype=torch.float64), n)
        raise AssertionError("A verificação de estouro deveria ter falhado")
    except ValueError:
        print(f"  Estouro detectado antes de compartilhar (limite por cliente {SMPC_LIMITE_SOMA / n:.2e}) OK")

    p_antigo, escala_antiga = 2147483647, 2 ** 24
    soma_antiga = 3 * round(30.0 * escala_antiga) % p_antigo
    soma_antiga = soma_antiga - p_antigo if soma_antiga > p_antigo // 2 else soma_antiga
    print(f"  Configuração antiga (p = 2^31-1): 3 clientes com peso 30 -> média {soma_antiga / escala_antiga / 3:+.2f} "
          f"(esperado +30.00): estouro silencioso corrigido.")


autoteste_smpc()
```

```
--- Autoteste do SMPC ---
  Reconstrução com negativos e valores grandes: erro máx. 3.02e-08 (tolerância 8.94e-08) OK
  Uma parte isolada decodificada parece ruído: [31571435814.74223, -37256027090.85378, 66708676895.57681]
  Estouro detectado antes de compartilhar (limite por cliente 2.29e+10) OK
  Configuração antiga (p = 2^31-1): 3 clientes com peso 30 -> média -12.67 (esperado +30.00): estouro silencioso corrigido.
```

```
# ======================================================================
# --- 2. DADOS (engenharia de features do Teste 4, sem data leakage) ---
# ======================================================================

def ler_base(nome):
    for caminho in CAMINHOS[nome]:
        if os.path.exists(caminho):
            print(f"Carregando '{caminho}'...")
            return pd.read_csv(caminho)
    return None


def amostrar(df, alvo):
    """Mantém todas as fraudes e RAZAO_NORMAIS_POR_FRAUDE vezes mais transações normais."""
    fraudes = df[df[alvo] == 1]
    normais = df[df[alvo] == 0]
    normais = normais.sample(n=min(len(fraudes) * RAZAO_NORMAIS_POR_FRAUDE, len(normais)), random_state=42)
    return pd.concat([normais, fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)


def preparar_creditcard(df):
    df = amostrar(df.dropna(subset=['Class']), 'Class')
    df = df.drop(columns=['Time'], errors='ignore')      # Contador monotônico: ruído para o GAN
    df['amount_log'] = np.log1p(df['Amount'].clip(lower=0))
    return df.drop(columns=['Amount']), 'Class'


def preparar_paysim(df):
    df = amostrar(df.dropna(subset=['isFraud']), 'isFraud')
    df['orig_balance_zero'] = (df['oldbalanceOrg'] == 0).astype(int)
    df['dest_balance_zero'] = (df['oldbalanceDest'] == 0).astype(int)
    for col in ['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldbalanceOrg']
    df['errorBalanceDest'] = df['oldbalanceDest'] + df['amount'] - df['newbalanceDest']
    df = df.drop(columns=['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest'])
    df = df.drop(columns=[c for c in ['nameOrig', 'nameDest', 'isFlaggedFraud'] if c in df.columns])
    return pd.get_dummies(df, columns=['type'], drop_first=True, dtype=float), 'isFraud'


PREPARADORES = {'creditcard': preparar_creditcard, 'paysim': preparar_paysim}
```

```
def carregar_dados(nome):
    df = ler_base(nome)
    if df is None:
        return None
    df, alvo = PREPARADORES[nome](df)
    X = df.drop(columns=[alvo]).values.astype(np.float32)
    y = df[alvo].values.astype(np.int64)

    # Split 60/20/20 estratificado ANTES de escalar
    X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

    # MinMaxScaler(-1, 1) alinhado ao Tanh dos geradores, ajustado SÓ no treino
    scaler = MinMaxScaler(feature_range=(-1, 1))
    X_train = scaler.fit_transform(X_train).astype(np.float32)
    X_val = scaler.transform(X_val).astype(np.float32)
    X_test = scaler.transform(X_test).astype(np.float32)

    # Silos bancários: fatias do treino
    fatias = np.array_split(np.arange(len(X_train)), NUM_CLIENTES)
    clientes = [(X_train[i], y_train[i]) for i in fatias]
    pesos_fedavg = [len(i) / len(X_train) for i in fatias]

    print(f"[DADOS] Features: {X_train.shape[1]} | Treino: {len(X_train)} (fraudes: {int(y_train.sum())}) "
          f"| Validação: {len(X_val)} (fraudes: {int(y_val.sum())}) | Teste: {len(X_test)} (fraudes: {int(y_test.sum())})")
    return dict(nome=nome, input_dim=X_train.shape[1], clientes=clientes, pesos_fedavg=pesos_fedavg,
                X_val=X_val, y_val=y_val, X_test=X_test, y_test=y_test)
```

```
# ======================================================================
# --- 3. STEP-GAN: GERADORES CONDICIONADOS POR LIMIARES (comentário 3) ---
# ======================================================================

class GeradorCondicional(nn.Module):
    """G_k(z, τ): gera pseudo-anomalias cuja saída no Discriminador deve ficar em τ (limiar do degrau k)."""

    def __init__(self, input_dim, latent_dim=HP_LATENT_DIM):
        super().__init__()
        self.condicao = nn.Sequential(nn.Linear(1, DIM_CONDICAO), nn.LeakyReLU(0.2))
        self.net = nn.Sequential(
            nn.Linear(latent_dim + DIM_CONDICAO, 128), nn.BatchNorm1d(128), nn.LeakyReLU(0.2),
            nn.Linear(128, 256), nn.BatchNorm1d(256), nn.LeakyReLU(0.2),
            nn.Linear(256, 512), nn.BatchNorm1d(512), nn.LeakyReLU(0.2),
            nn.Linear(512, input_dim), nn.Tanh()
        )

    def forward(self, z, tau):
        return self.net(torch.cat([z, self.condicao(tau)], dim=1))


class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 512), nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(512, 256), nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(256, 64), nn.LeakyReLU(0.2), nn.Dropout(0.2),
            nn.Linear(64, 1), nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)


def amostrar_tau(k, n):
    """Limiar τ ~ U(degrau k): a condição do gerador k."""
    minimo, maximo = DEGRAUS[k]
    return torch.rand(n, 1, device=device) * (maximo - minimo) + minimo


def custom_bce_loss(pred, target, real_labels_flag):
    """BCE com peso condicional: erro em fraude real pesa PESO_FRAUDE vezes mais."""
    loss = nn.BCELoss(reduction='none')(pred, target)
    weights = torch.ones_like(target)
    weights[(target < 0.5) & real_labels_flag] = PESO_FRAUDE
    return (loss * weights).mean()


def novo_estado_global(input_dim):
    return {'D': copy.deepcopy(Discriminator(input_dim).state_dict()),
            'G': [copy.deepcopy(GeradorCondicional(input_dim).state_dict()) for _ in DEGRAUS]}
```

```
def treinar_local(estado_global, dados_X, dados_y, input_dim, semente):
    """Treino local do STEP-GAN a partir do modelo global (D + K geradores)."""
    torch.manual_seed(semente)
    K = len(DEGRAUS)
    D = Discriminator(input_dim).to(device)
    D.load_state_dict(estado_global['D'])
    Gs = []
    for estado_G in estado_global['G']:
        G = GeradorCondicional(input_dim).to(device)
        G.load_state_dict(estado_G)
        Gs.append(G)
    D.train()
    for G in Gs:
        G.train()

    opt_D = optim.Adam(D.parameters(), lr=HP_LR_D, betas=(0.5, 0.999), weight_decay=HP_WEIGHT_DECAY)
    opts_G = [optim.Adam(G.parameters(), lr=HP_LR_G, betas=(0.5, 0.999), weight_decay=HP_WEIGHT_DECAY) for G in Gs]
    loader = DataLoader(TensorDataset(torch.from_numpy(dados_X), torch.from_numpy(dados_y).float()),
                        batch_size=HP_BATCH_SIZE, shuffle=True, generator=torch.Generator().manual_seed(semente))

    for _ in range(EPOCAS_LOCAIS):
        for i, (real_data, real_y) in enumerate(loader):
            real_data, real_y = real_data.to(device), real_y.to(device)
            batch_s = real_data.size(0)
            n_k = max(batch_s // K, 2)                   # BatchNorm exige pelo menos 2 amostras
            if batch_s < 2:
                continue

            real_labels = torch.where(real_y.unsqueeze(1) == 0,
                                      torch.tensor(ALVO_NORMAL, device=device),
                                      torch.tensor(ALVO_ANOMALIA, device=device))
            taus = [amostrar_tau(k, n_k) for k in range(K)]
            fakes = [G(torch.randn(n_k, HP_LATENT_DIM, device=device), tau) for G, tau in zip(Gs, taus)]

            # --- Discriminador graduado: normal real -> 0.9; fraude real -> 0.1; amostra do degrau τ -> τ ---
            opt_D.zero_grad()
            loss_real = custom_bce_loss(D(real_data), real_labels, torch.ones_like(real_labels, dtype=torch.bool))
            fake_cat = torch.cat([f.detach() for f in fakes])
            if DISCRIMINADOR_GRADUADO:
                fake_labels = torch.cat(taus).sub(MARGEM_GRADUADA).clamp(min=ALVO_ANOMALIA)
            else:
                fake_labels = torch.full((fake_cat.size(0), 1), ALVO_ANOMALIA, device=device)
            loss_fake = custom_bce_loss(D(fake_cat), fake_labels, torch.zeros_like(fake_labels, dtype=torch.bool))
            (loss_real + loss_fake).backward()
            torch.nn.utils.clip_grad_norm_(D.parameters(), max_norm=1.0)
            opt_D.step()

            # --- Geradores (a cada 2 passos do D): condicionamento D(G_k(z, τ)) -> τ ---
            if i % 2 == 0:
                for G, opt_G, fake, tau in zip(Gs, opts_G, fakes, taus):
                    opt_G.zero_grad()
                    loss_G = nn.BCELoss()(D(fake), tau)
                    loss_G.backward()
                    torch.nn.utils.clip_grad_norm_(G.parameters(), max_norm=1.0)
                    opt_G.step()

    return {'D': {k: v.detach().cpu().clone() for k, v in D.state_dict().items()},
            'G': [{k: v.detach().cpu().clone() for k, v in G.state_dict().items()} for G in Gs]}
```

```
# ======================================================================
# --- 4. APRENDIZADO FEDERADO + AVALIAÇÃO ---
# ======================================================================

def scores_anomalia(estado_D, input_dim, X, tamanho_lote=65536):
    D = Discriminator(input_dim).to(device)
    D.load_state_dict(estado_D)
    D.eval()
    saida = []
    with torch.no_grad():
        for i in range(0, len(X), tamanho_lote):
            saida.append(D(torch.from_numpy(X[i:i + tamanho_lote]).to(device)).cpu().numpy().ravel())
    return 1.0 - np.concatenate(saida)        # D ≈ 0.9 normal, ≈ 0.1 anomalia


def escolher_limiar(y_val, s_val):
    """Maior recall com precisão >= PRECISAO_MINIMA na VALIDAÇÃO; se inalcançável, máximo F1."""
    precisions, recalls, thresholds = precision_recall_curve(y_val, s_val)
    validos = np.where(precisions[:-1] >= PRECISAO_MINIMA)[0]
    if len(validos) > 0:
        return thresholds[validos[np.argmax(recalls[validos])]]
    f1 = np.divide(2 * precisions * recalls, precisions + recalls,
                   out=np.zeros_like(precisions), where=(precisions + recalls) != 0)
```

```
    return thresholds[np.argmax(f1[:-1])]


def executar_federado(dados, semente):
    """FL com NUM_RODADAS rodadas. D e os K geradores são agregados via SMPC. Retorna o global da melhor rodada."""
    input_dim = dados['input_dim']
    seed_everything(semente)
    estado_global = novo_estado_global(input_dim)
    historico, melhor = [], None

    for rodada in range(1, NUM_RODADAS + 1):
        sincronizar()
        t0 = time.perf_counter()
        locais = [treinar_local(estado_global, X_k, y_k, input_dim, semente * 1000 + rodada * 10 + k)
                  for k, (X_k, y_k) in enumerate(dados['clientes'])]
        sincronizar()
        t_treino = time.perf_counter() - t0

        # Um pacote por cliente com D e todos os geradores: uma única agregação SMPC por rodada
        pacotes = [{**{f'D.{c}': t for c, t in l['D'].items()},
                    **{f'G{k}.{c}': t for k, g in enumerate(l['G']) for c, t in g.items()}} for l in locais]
        agregado, custo = agregar_smpc(pacotes, dados['pesos_fedavg'])
        estado_global = {'D': {c[2:]: t for c, t in agregado.items() if c.startswith('D.')},
                         'G': [{c[len(f'G{k}.'):]: t for c, t in agregado.items() if c.startswith(f'G{k}.')}
                               for k in range(len(DEGRAUS))]}

        pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado_global['D'], input_dim, dados['X_val']))
        historico.append(dict(rodada=rodada, pr_auc_val=pr_auc_val, t_treino=t_treino, **custo))
        print(f"  Rodada {rodada}/{NUM_RODADAS} | PR-AUC val {pr_auc_val:.4f} | treino {t_treino:.1f}s "
              f"| SMPC {1000 * custo['t_smpc']:.0f} ms (erro máx. vs claro {custo['erro_max']:.1e})")
        if melhor is None or pr_auc_val > melhor[0]:
            melhor = (pr_auc_val, rodada, copy.deepcopy(estado_global))

    return melhor[2], melhor[1], pd.DataFrame(historico)


def avaliar(estado_D, dados):
    input_dim = dados['input_dim']
    limiar = escolher_limiar(dados['y_val'], scores_anomalia(estado_D, input_dim, dados['X_val']))
    sincronizar()
    t0 = time.perf_counter()
    s = scores_anomalia(estado_D, input_dim, dados['X_test'])
    sincronizar()
    t_inf = time.perf_counter() - t0
    y, y_pred = dados['y_test'], (s >= limiar).astype(int)
    return dict(scores=s, y_pred=y_pred, limiar=limiar, **{
        'ROC-AUC': roc_auc_score(y, s), 'PR-AUC': average_precision_score(y, s),
        'Precisão': precision_score(y, y_pred, zero_division=0), 'Recall': recall_score(y, y_pred, zero_division=0),
        'F1': f1_score(y, y_pred, zero_division=0), 'Acurácia': accuracy_score(y, y_pred),
        'TPS': len(y) / t_inf if t_inf > 0 else float('nan')})


def aderencia_condicionamento(estado_global, input_dim, n=2000):
    """Mede se cada gerador respeita seu limiar: compara τ pedido com D(G_k(z, τ)) obtido."""
    D = Discriminator(input_dim).to(device)
    D.load_state_dict(estado_global['D'])
    D.eval()
    resultado = []
    with torch.no_grad():
        for k, estado_G in enumerate(estado_global['G']):
            G = GeradorCondicional(input_dim).to(device)
            G.load_state_dict(estado_G)
            G.eval()
            tau = amostrar_tau(k, n)
            saida = D(G(torch.randn(n, HP_LATENT_DIM, device=device), tau))
            resultado.append(pd.DataFrame({'gerador': f'G{k + 1} {DEGRAUS[k]}',
                                           'tau': tau.cpu().numpy().ravel(), 'D': saida.cpu().numpy().ravel()}))
    return pd.concat(resultado, ignore_index=True)
```

```
# ======================================================================
# --- 5. EXPERIMENTO: CENTRALIZADO × FEDERADO + SMPC (Teste 6) ---
# ======================================================================
from sklearn.metrics import roc_curve

METRICA_PRINCIPAL = 'ROC-AUC'       # Métrica do teste de não-inferioridade ('ROC-AUC' ou 'PR-AUC')
MARGEM_NAO_INFERIORIDADE = 0.01     # Perda máxima aceitável do federado em relação ao centralizado
N_BOOTSTRAP = 300                   # Reamostragens do teste para o IC 95% do Δ médio

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CENTRAL, FEDERADO, LOCAL = 'Centralizado', 'Federado + SMPC', 'Local isolado'
CONFIGS = [CENTRAL, FEDERADO, LOCAL]
CORES = {CENTRAL: '#1f77b4', FEDERADO: '#d62728', LOCAL: '#7f7f7f'}
```

```
def treinar_em_blocos(dados, X, y, semente, id_semente_bloco):
    """
    Treino SEM federação com o mesmo procedimento do FL: parte dos mesmos pesos iniciais, treina NUM_RODADAS blocos
    de EPOCAS_LOCAIS épocas e guarda o checkpoint com melhor PR-AUC na validação.
    """
    input_dim = dados['input_dim']
    seed_everything(semente)
    estado = novo_estado_global(input_dim)          # Mesma inicialização de executar_federado()
    historico, melhor, t_total = [], None, 0.0
    for bloco in range(1, NUM_RODADAS + 1):
        sincronizar()
        t0 = time.perf_counter()
        estado = treinar_local(estado, X, y, input_dim, semente * 1000 + bloco * 10 + id_semente_bloco)
        sincronizar()
        t_total += time.perf_counter() - t0
        pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado['D'], input_dim, dados['X_val']))
        historico.append(dict(rodada=bloco, pr_auc_val=pr_auc_val))
        if melhor is None or pr_auc_val > melhor[0]:
            melhor = (pr_auc_val, bloco, copy.deepcopy(estado))
    return melhor[2], melhor[1], pd.DataFrame(historico), t_total


def bootstrap_delta(y, pares_scores, n_boot, semente):
    """IC 95% do Δ médio entre sementes (federado − centralizado), reamostrando o conjunto de teste."""
    rng = np.random.default_rng(semente)
    deltas = {'ROC-AUC': [], 'PR-AUC': []}
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
        if y[idx].min() == y[idx].max():
            continue
        for m, fn in [('ROC-AUC', roc_auc_score), ('PR-AUC', average_precision_score)]:
            deltas[m].append(np.mean([fn(y[idx], f[idx]) - fn(y[idx], c[idx]) for c, f in pares_scores]))
    return {m: np.array(v) for m, v in deltas.items()}


resultados_gerais = {}

for nome_base in DATASETS:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue
    X_central = np.concatenate([c[0] for c in dados['clientes']])   # União exata dos dados dos clientes
    y_central = np.concatenate([c[1] for c in dados['clientes']])

    linhas, tempos, pares_scores, curvas, exemplo = [], [], [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        # (a) Centralizado
        est_c, bloco_c, hist_c, t_c = treinar_em_blocos(dados, X_central, y_central, semente, id_semente_bloco=9)
        print(f"[Centralizado] treino {t_c:.1f}s | melhor bloco {bloco_c} | PR-AUC val {hist_c['pr_auc_val'].max():.4f}")

        # (b) Federado + SMPC (modelo final)
        est_f, rodada_f, hist_f = executar_federado(dados, semente)
        t_f = hist_f['t_treino'].sum()

        # (c) Local isolado: cada cliente com as mesmas sementes que usa no federado
        avs_locais = []
        for k, (X_k, y_k) in enumerate(dados['clientes']):
            est_k, _, _, _ = treinar_em_blocos(dados, X_k, y_k, semente, id_semente_bloco=k)
            avs_locais.append(avaliar(est_k['D'], dados))
        print(f"[Local isolado] PR-AUC teste por cliente: {[round(a['PR-AUC'], 4) for a in avs_locais]}")

        av = {CENTRAL: avaliar(est_c['D'], dados), FEDERADO: avaliar(est_f['D'], dados),
              LOCAL: {m: np.mean([a[m] for a in avs_locais]) for m in METRICAS}}
        for cfg in CONFIGS:
            linhas.append({'Semente': semente, 'Configuração': cfg, **{m: av[cfg][m] for m in METRICAS}})
            print(f"  {cfg:16s} ROC-AUC {av[cfg]['ROC-AUC']:.4f} | PR-AUC {av[cfg]['PR-AUC']:.4f} | F1 {av[cfg]['F1']:.4f}'
        tempos.append({'Semente': semente, 'Centralizado': t_c, 'Federado (sequencial)': t_f,
                       'Federado (paralelo, estimado)': t_f / NUM_CLIENTES,
                       'SMPC total': hist_f['t_smpc'].sum(), 'Erro máx. SMPC': hist_f['erro_max'].max(),
                       'Bloco escolhido (central)': bloco_c, 'Rodada escolhida (federado)': rodada_f})
        pares_scores.append((av[CENTRAL]['scores'], av[FEDERADO]['scores']))
        curvas.append((hist_c.assign(semente=semente, cfg=CENTRAL), hist_f[['rodada', 'pr_auc_val']].assign(semente=semente
        if exemplo is None:
            exemplo = {cfg: av[cfg]['scores'] for cfg in [CENTRAL, FEDERADO]}
```

```
df = pd.DataFrame(linhas)
df_tempos = pd.DataFrame(tempos).set_index('Semente')
df_curvas = pd.concat([pd.concat(c) for c in curvas], ignore_index=True)

# ----- Resumo -----
resumo = df.groupby('Configuração')[METRICAS].agg(['mean', 'std']).reindex(CONFIGS)
print(f"\n>> MÉTRICAS NO TESTE: média ± desvio-padrão em {len(SEMENTES)} sementes")
print(pd.DataFrame({m: [f"{resumo.loc[c, (m, 'mean')]:.4f} ± {np.nan_to_num(resumo.loc[c, (m, 'std')]):.4f}"
                        for c in CONFIGS] for m in METRICAS}, index=CONFIGS).to_string())

piv = df.pivot(index='Semente', columns='Configuração')
deltas = {m: (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).values for m in METRICAS}
print(f"\n>> Δ PAREADO POR SEMENTE ({FEDERADO} − {CENTRAL}):")
for m in METRICAS:
    d = deltas[m]
    meia = stats.t.ppf(0.975, len(d) - 1) * d.std(ddof=1) / np.sqrt(len(d)) if len(d) > 1 else 0.0
    print(f"  {m:9s}: {d.mean():+.4f}  (IC95% entre sementes: [{d.mean() - meia:+.4f}, {d.mean() + meia:+.4f}])")

boot = bootstrap_delta(dados['y_test'], pares_scores, N_BOOTSTRAP, SEMENTES[0])
ic = {m: np.percentile(v, [2.5, 97.5]) for m, v in boot.items()}
print(f"\n>> IC 95% POR BOOTSTRAP DO Δ MÉDIO ({N_BOOTSTRAP} reamostragens do teste):")
for m in ['ROC-AUC', 'PR-AUC']:
    veredito = 'não-inferior' if ic[m][0] > -MARGEM_NAO_INFERIORIDADE else 'não-inferioridade NÃO demonstrada'
    print(f"  Δ{m}: médio {deltas[m].mean():+.4f} | IC95% [{ic[m][0]:+.4f}, {ic[m][1]:+.4f}] -> {veredito} "
          f"(margem {MARGEM_NAO_INFERIORIDADE})")

nao_inferior = ic[METRICA_PRINCIPAL][0] > -MARGEM_NAO_INFERIORIDADE
print(f"\n>> RESPOSTA À QUESTÃO DE PESQUISA (métrica principal: {METRICA_PRINCIPAL}):")
if nao_inferior:
    print(f"  O STEP-GAN Federado + SMPC é NÃO-INFERIOR ao baseline centralizado: limite inferior do IC95% "
          f"{ic[METRICA_PRINCIPAL][0]:+.4f} > {-MARGEM_NAO_INFERIORIDADE}.")
else:
    print(f"  NÃO foi possível demonstrar não-inferioridade: limite inferior do IC95% "
          f"{ic[METRICA_PRINCIPAL][0]:+.4f} <= {-MARGEM_NAO_INFERIORIDADE}.")
ganho = (piv[(METRICA_PRINCIPAL, FEDERADO)] - piv[(METRICA_PRINCIPAL, LOCAL)]).mean()
print(f"  Ganho da colaboração ({FEDERADO} − {LOCAL}): {ganho:+.4f} {METRICA_PRINCIPAL}")
print(f"  SMPC: erro máx. vs FedAvg em claro {df_tempos['Erro máx. SMPC'].max():.1e}")

print("\n>> TEMPOS (s) E CHECKPOINTS ESCOLHIDOS POR SEMENTE:")
print(df_tempos.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}"}, float_format=lambda x: f"{x:.2f}"))

arquivo = f'resultados_teste6_{nome_base}.csv'
df.to_csv(arquivo, index=False)
print(f"Resultados por semente salvos em '{arquivo}'.")
resultados_gerais[nome_base] = dict(metricas=df, tempos=df_tempos, ic_bootstrap=ic, nao_inferior=nao_inferior)

# ----- Visualizações -----
fig, axes = plt.subplots(2, 3, figsize=(22, 11))
fig.suptitle(f"Teste 6 (v2): Centralizado × Federado + SMPC (STEP-GAN v5, {nome_base}, {len(SEMENTES)} sementes)",
             fontsize=14, fontweight='bold')

# 1. Métricas
ax = axes[0, 0]
largura = 0.8 / len(CONFIGS)
x = np.arange(len(METRICAS))
for i, cfg in enumerate(CONFIGS):
    ax.bar(x + (i - (len(CONFIGS) - 1) / 2) * largura, [resumo.loc[cfg, (m, 'mean')] for m in METRICAS], largura,
           yerr=[np.nan_to_num(resumo.loc[cfg, (m, 'std')]) for m in METRICAS], capsize=3,
           color=CORES[cfg], label=cfg)
ax.set_xticks(x)
ax.set_xticklabels(METRICAS)
ax.set_ylim(0, 1.2)
ax.set_title('Métricas no teste (média ± desvio)', fontweight='bold')
ax.legend(loc='upper center', ncol=3)

# 2. Curvas ROC (primeira semente)
ax = axes[0, 1]
for cfg in [CENTRAL, FEDERADO]:
    fpr, tpr, _ = roc_curve(dados['y_test'], exemplo[cfg])
    ax.plot(fpr, tpr, '-' if cfg == CENTRAL else '--', color=CORES[cfg],
            label=f"{cfg} (AUC={roc_auc_score(dados['y_test'], exemplo[cfg]):.4f})")
ax.plot([0, 1], [0, 1], ':', color='gray')
ax.set_title(f'Curva ROC no teste (semente {SEMENTES[0]})', fontweight='bold')
ax.set_xlabel('FPR')
ax.set_ylabel('TPR')
ax.legend(loc='lower right')

# 3. Convergência: PR-AUC na validação por bloco/rodada
ax = axes[0, 2]
for cfg in [CENTRAL, FEDERADO]:
    sub = df_curvas[df_curvas['cfg'] == cfg]
```

```
            for _, g in sub.groupby('semente'):
                ax.plot(g['rodada'], g['pr_auc_val'], color=CORES[cfg], alpha=0.25)
            media = sub.groupby('rodada')['pr_auc_val'].mean()
            ax.plot(media.index, media.values, color=CORES[cfg], lw=2.5, marker='o', label=f'{cfg} (média)')
        ax.set_xlabel(f'Rodada / bloco de {EPOCAS_LOCAIS} épocas')
        ax.set_title('PR-AUC na validação', fontweight='bold')
        ax.legend()

        # 4. Δ por semente e IC de não-inferioridade
        ax = axes[1, 0]
        for i, m in enumerate(['ROC-AUC', 'PR-AUC', 'F1']):
            ax.scatter(np.full(len(deltas[m]), i), deltas[m], color=CORES[FEDERADO], zorder=3,
                       label='Δ por semente' if i == 0 else None)
            ax.plot([i - 0.2, i + 0.2], [deltas[m].mean()] * 2, color='black', lw=2, label='Δ médio' if i == 0 else None)
        for i, m in enumerate(['ROC-AUC', 'PR-AUC']):
            ax.errorbar(i + 0.3, np.mean(ic[m]), yerr=[[np.mean(ic[m]) - ic[m][0]], [ic[m][1] - np.mean(ic[m])]],
                        fmt='none', ecolor='#2ca02c', capsize=6, lw=2, label='IC95% bootstrap' if i == 0 else None)
        ax.axhline(0, color='gray', lw=1)
        ax.axhline(-MARGEM_NAO_INFERIORIDADE, color='#d62728', ls='--',
                   label=f'Margem de não-inferioridade (−{MARGEM_NAO_INFERIORIDADE})')
        ax.set_xticks(range(3))
        ax.set_xticklabels(['ΔROC-AUC', 'ΔPR-AUC', 'ΔF1'])
        ax.set_title(f'{FEDERADO} − {CENTRAL}', fontweight='bold')
        ax.legend(loc='best', fontsize=9)

        # 5. Tempo de treino
        ax = axes[1, 1]
        media_t = df_tempos[['Centralizado', 'Federado (sequencial)', 'Federado (paralelo, estimado)']].mean()
        ax.bar(media_t.index, media_t.values, color=['#1f77b4', '#ff9896', '#d62728'])
        for i, v in enumerate(media_t.values):
            ax.text(i, v, f"{v:.1f}s", ha='center', va='bottom')
        ax.set_title(f"Tempo de treino (SMPC total: {df_tempos['SMPC total'].mean():.2f}s)", fontweight='bold')
        ax.set_ylabel('segundos')

        axes[1, 2].axis('off')
        plt.tight_layout()
        plt.show()

    print("\n[SUCESSO] Teste 6 (v2) concluído!")
```

```
================================================================================
 BASE: PAYSIM | 5 sementes | 5 x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv'...
[DADOS] Features: 14 | Treino: 990487 (fraudes: 4927) | Validação: 330163 (fraudes: 1643) | Teste: 330163 (fraudes: 1643)

--- Semente 42 ---
[Centralizado] treino 1027.1s | melhor bloco 5 | PR-AUC val 0.9558
  Rodada 1/5 | PR-AUC val 0.8765 | treino 203.5s | SMPC 393 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8944 | treino 203.7s | SMPC 351 ms (erro máx. vs claro 8.8e-08)
  Rodada 3/5 | PR-AUC val 0.9099 | treino 203.4s | SMPC 360 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9224 | treino 205.7s | SMPC 359 ms (erro máx. vs claro 8.8e-08)
  Rodada 5/5 | PR-AUC val 0.9280 | treino 204.2s | SMPC 350 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9286), np.float64(0.9236), np.float64(0.9307)]
  Centralizado     ROC-AUC 0.9995 | PR-AUC 0.9619 | F1 0.8106
  Federado + SMPC  ROC-AUC 0.9992 | PR-AUC 0.9309 | F1 0.7838
  Local isolado    ROC-AUC 0.9991 | PR-AUC 0.9276 | F1 0.7824

--- Semente 43 ---
[Centralizado] treino 999.8s | melhor bloco 5 | PR-AUC val 0.9515
  Rodada 1/5 | PR-AUC val 0.8681 | treino 200.0s | SMPC 501 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8859 | treino 199.5s | SMPC 348 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8975 | treino 200.6s | SMPC 355 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9168 | treino 200.8s | SMPC 367 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9243 | treino 200.0s | SMPC 346 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9273), np.float64(0.9123), np.float64(0.9264)]
  Centralizado     ROC-AUC 0.9996 | PR-AUC 0.9557 | F1 0.8099
  Federado + SMPC  ROC-AUC 0.9992 | PR-AUC 0.9249 | F1 0.7760
  Local isolado    ROC-AUC 0.9991 | PR-AUC 0.9220 | F1 0.7784

--- Semente 44 ---
[Centralizado] treino 996.3s | melhor bloco 5 | PR-AUC val 0.9515
  Rodada 1/5 | PR-AUC val 0.8722 | treino 200.2s | SMPC 340 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8926 | treino 200.5s | SMPC 347 ms (erro máx. vs claro 8.8e-08)
  Rodada 3/5 | PR-AUC val 0.9096 | treino 201.1s | SMPC 347 ms (erro máx. vs claro 8.8e-08)
  Rodada 4/5 | PR-AUC val 0.9204 | treino 200.9s | SMPC 343 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9244 | treino 203.7s | SMPC 383 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9251), np.float64(0.9263), np.float64(0.9252)]
  Centralizado     ROC-AUC 0.9994 | PR-AUC 0.9561 | F1 0.8069
  Federado + SMPC  ROC-AUC 0.9991 | PR-AUC 0.9294 | F1 0.7825
  Local isolado    ROC-AUC 0.9990 | PR-AUC 0.9255 | F1 0.7818

--- Semente 45 ---
[Centralizado] treino 998.9s | melhor bloco 5 | PR-AUC val 0.9517
  Rodada 1/5 | PR-AUC val 0.8664 | treino 200.0s | SMPC 340 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8862 | treino 199.6s | SMPC 348 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.9024 | treino 199.9s | SMPC 343 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9180 | treino 201.7s | SMPC 345 ms (erro máx. vs claro 8.8e-08)
  Rodada 5/5 | PR-AUC val 0.9269 | treino 201.1s | SMPC 343 ms (erro máx. vs claro 8.8e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9244), np.float64(0.9169), np.float64(0.929)]
  Centralizado     ROC-AUC 0.9995 | PR-AUC 0.9572 | F1 0.8092
  Federado + SMPC  ROC-AUC 0.9992 | PR-AUC 0.9256 | F1 0.7787
  Local isolado    ROC-AUC 0.9991 | PR-AUC 0.9234 | F1 0.7783

--- Semente 46 ---
[Centralizado] treino 992.8s | melhor bloco 5 | PR-AUC val 0.9478
  Rodada 1/5 | PR-AUC val 0.8672 | treino 198.4s | SMPC 363 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8852 | treino 198.3s | SMPC 367 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8998 | treino 199.2s | SMPC 345 ms (erro máx. vs claro 8.8e-08)
  Rodada 4/5 | PR-AUC val 0.9173 | treino 199.4s | SMPC 338 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9197 | treino 199.1s | SMPC 358 ms (erro máx. vs claro 8.8e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9152), np.float64(0.9116), np.float64(0.9244)]
  Centralizado     ROC-AUC 0.9991 | PR-AUC 0.9498 | F1 0.7985
  Federado + SMPC  ROC-AUC 0.9992 | PR-AUC 0.9183 | F1 0.7752
  Local isolado    ROC-AUC 0.9990 | PR-AUC 0.9171 | F1 0.7735

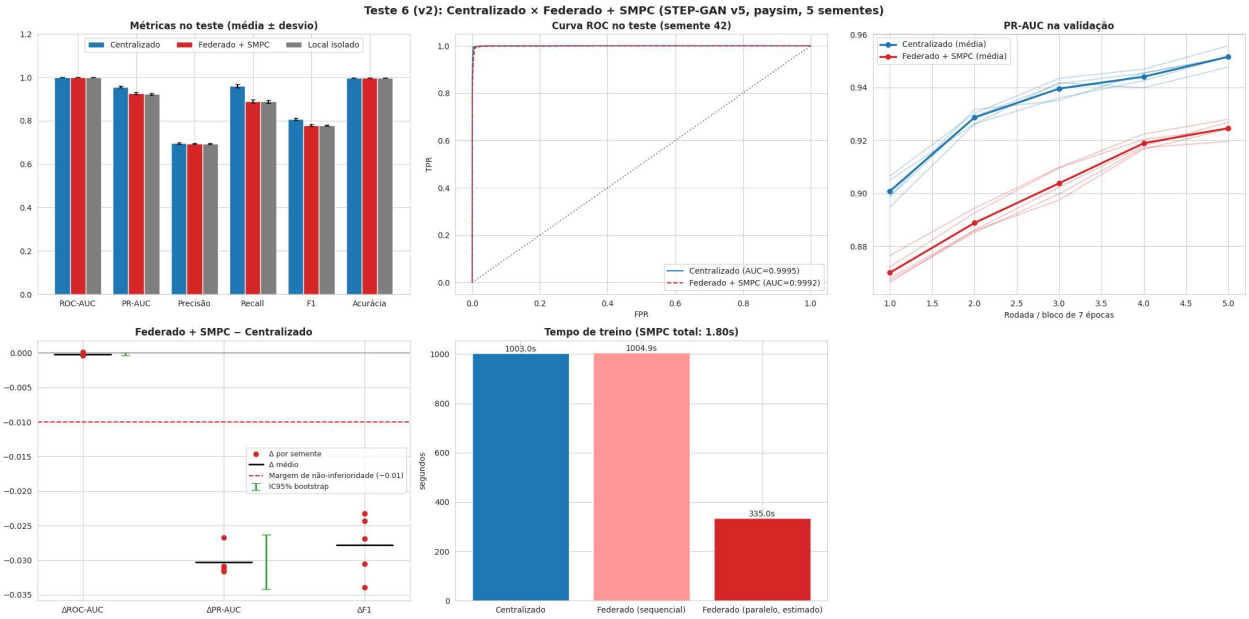
>> MÉTRICAS NO TESTE: média ± desvio-padrão em 5 sementes
                         ROC-AUC           PR-AUC         Precisão           Recall               F1         Acurácia
Centralizado     0.9994 ± 0.0002  0.9562 ± 0.0043  0.6956 ± 0.0041  0.9609 ± 0.0087  0.8070 ± 0.0050  0.9977 ± 0.0001
Federado + SMPC  0.9992 ± 0.0000  0.9258 ± 0.0049  0.6936 ± 0.0031  0.8890 ± 0.0076  0.7792 ± 0.0038  0.9975 ± 0.0000
Local isolado    0.9991 ± 0.0000  0.9231 ± 0.0040  0.6934 ± 0.0024  0.8884 ± 0.0067  0.7789 ± 0.0036  0.9975 ± 0.0000

>> Δ PAREADO POR SEMENTE (Federado + SMPC − Centralizado):
  ROC-AUC  : -0.0002  (IC95% entre sementes: [-0.0005, -0.0000])
  PR-AUC   : -0.0303  (IC95% entre sementes: [-0.0329, -0.0278])
  Precisão : -0.0020  (IC95% entre sementes: [-0.0078, +0.0038])
  Recall   : -0.0719  (IC95% entre sementes: [-0.0826, -0.0613])
  F1       : -0.0278  (IC95% entre sementes: [-0.0333, -0.0223])
  Acurácia : -0.0002  (IC95% entre sementes: [-0.0003, -0.0002])

>> IC 95% POR BOOTSTRAP DO Δ MÉDIO (300 reamostragens do teste):
  ΔROC-AUC: médio -0.0002 | IC95% [-0.0004, -0.0000] -> não-inferior (margem 0.01)
  ΔPR-AUC: médio -0.0303 | IC95% [-0.0341, -0.0263] -> não-inferioridade NÃO demonstrada (margem 0.01)

>> RESPOSTA À QUESTÃO DE PESQUISA (métrica principal: ROC-AUC):
  O STEP-GAN Federado + SMPC é NÃO-INFERIOR ao baseline centralizado: limite inferior do IC95% -0.0004 > -0.01.
  Ganho da colaboração (Federado + SMPC − Local isolado): +0.0001 ROC-AUC
  SMPC: erro máx. vs FedAvg em claro 8.9e-08
```

```
>> TEMPOS (s) E CHECKPOINTS ESCOLHIDOS POR SEMENTE:
         Centralizado  Federado (sequencial)  Federado (paralelo, estimado)  SMPC total Erro máx. SMPC  Bloco escolhido (c
Semente
42            1027.14                1020.47                         340.16        1.81        8.9e-08
43             999.80                1001.06                         333.69        1.92        8.9e-08
44             996.33                1006.29                         335.43        1.76        8.9e-08
45             998.89                1002.26                         334.09        1.72        8.9e-08
46             992.79                 994.45                         331.48        1.77        8.9e-08
Resultados por semente salvos em 'resultados_teste6_paysim.csv'.
```




```
================================================================================
 BASE: CREDITCARD | 5 sementes | 5 x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv'...
[DADOS] Features: 29 | Treino: 59334 (fraudes: 295) | Validação: 19779 (fraudes: 99) | Teste: 19779 (fraudes: 98)

--- Semente 42 ---
[Centralizado] treino 58.6s | melhor bloco 5 | PR-AUC val 0.8461
  Rodada 1/5 | PR-AUC val 0.8092 | treino 12.1s | SMPC 363 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8250 | treino 12.3s | SMPC 354 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8315 | treino 12.3s | SMPC 353 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8330 | treino 12.2s | SMPC 354 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8354 | treino 12.1s | SMPC 360 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.8526), np.float64(0.8516), np.float64(0.8537)]
  Centralizado     ROC-AUC 0.9860 | PR-AUC 0.8631 | F1 0.8000
  Federado + SMPC  ROC-AUC 0.9832 | PR-AUC 0.8564 | F1 0.7887
  Local isolado    ROC-AUC 0.9817 | PR-AUC 0.8526 | F1 0.7952

--- Semente 43 ---
[Centralizado] treino 59.2s | melhor bloco 5 | PR-AUC val 0.8493
  Rodada 1/5 | PR-AUC val 0.8039 | treino 12.1s | SMPC 350 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8220 | treino 12.2s | SMPC 355 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8291 | treino 12.2s | SMPC 354 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8308 | treino 12.3s | SMPC 352 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8322 | treino 12.2s | SMPC 357 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.8493), np.float64(0.8511), np.float64(0.8516)]
  Centralizado     ROC-AUC 0.9854 | PR-AUC 0.8634 | F1 0.7925
  Federado + SMPC  ROC-AUC 0.9827 | PR-AUC 0.8536 | F1 0.7962
  Local isolado    ROC-AUC 0.9803 | PR-AUC 0.8507 | F1 0.7843

--- Semente 44 ---
[Centralizado] treino 60.1s | melhor bloco 5 | PR-AUC val 0.8466
  Rodada 1/5 | PR-AUC val 0.8202 | treino 12.2s | SMPC 365 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8315 | treino 12.2s | SMPC 369 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8335 | treino 12.4s | SMPC 358 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8340 | treino 12.0s | SMPC 483 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8342 | treino 12.1s | SMPC 517 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.8508), np.float64(0.853), np.float64(0.8521)]
  Centralizado     ROC-AUC 0.9857 | PR-AUC 0.8646 | F1 0.7962
  Federado + SMPC  ROC-AUC 0.9852 | PR-AUC 0.8528 | F1 0.7925
  Local isolado    ROC-AUC 0.9837 | PR-AUC 0.8520 | F1 0.7863

--- Semente 45 ---
[Centralizado] treino 59.9s | melhor bloco 5 | PR-AUC val 0.8347
  Rodada 1/5 | PR-AUC val 0.8105 | treino 11.5s | SMPC 502 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8227 | treino 12.0s | SMPC 542 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8293 | treino 12.0s | SMPC 353 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8304 | treino 12.2s | SMPC 356 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8309 | treino 12.3s | SMPC 352 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.8513), np.float64(0.8502), np.float64(0.8541)]
  Centralizado     ROC-AUC 0.9870 | PR-AUC 0.8612 | F1 0.8000
  Federado + SMPC  ROC-AUC 0.9851 | PR-AUC 0.8558 | F1 0.8038
  Local isolado    ROC-AUC 0.9825 | PR-AUC 0.8518 | F1 0.8003
```

```
--- Semente 46 ---
[Centralizado] treino 59.4s | melhor bloco 5 | PR-AUC val 0.8452
  Rodada 1/5 | PR-AUC val 0.8106 | treino 12.3s | SMPC 353 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8253 | treino 12.3s | SMPC 356 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8317 | treino 12.2s | SMPC 350 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8342 | treino 12.3s | SMPC 353 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8343 | treino 12.2s | SMPC 359 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.8515), np.float64(0.8512), np.float64(0.8532)]
  Centralizado     ROC-AUC 0.9860 | PR-AUC 0.8597 | F1 0.8077
  Federado + SMPC  ROC-AUC 0.9839 | PR-AUC 0.8540 | F1 0.8038
  Local isolado    ROC-AUC 0.9823 | PR-AUC 0.8520 | F1 0.7964

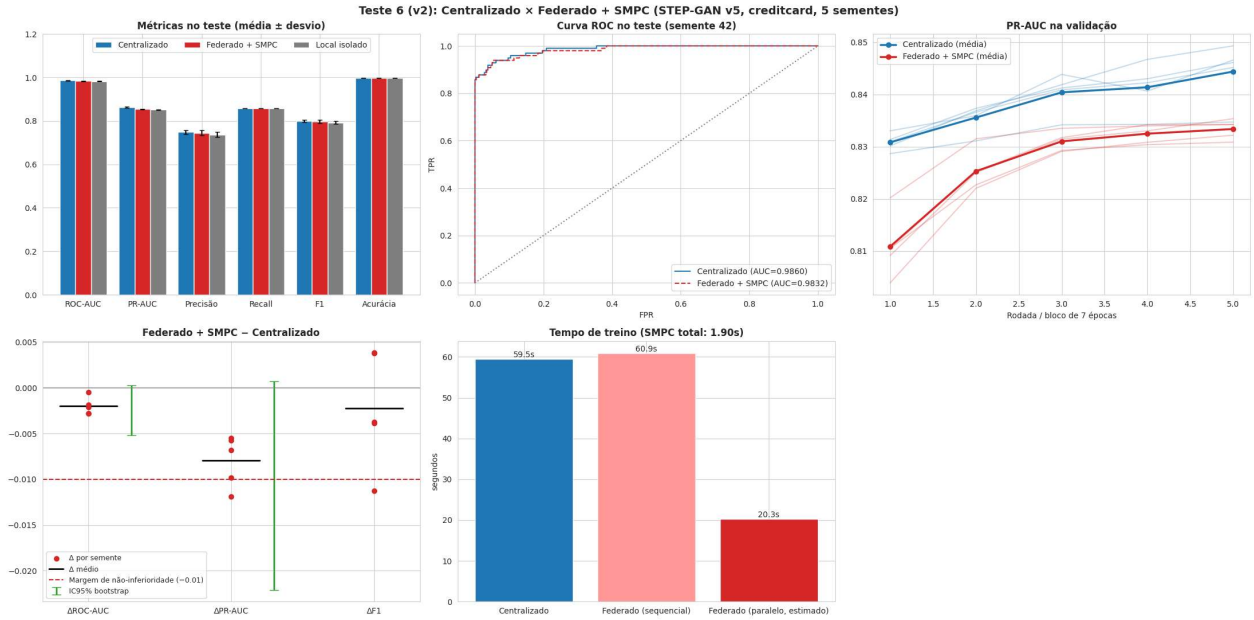
>> MÉTRICAS NO TESTE: média ± desvio-padrão em 5 sementes
                      ROC-AUC           PR-AUC         Precisão           Recall               F1         Acurácia
Centralizado     0.9860 ± 0.0006  0.8624 ± 0.0019  0.7488 ± 0.0099  0.8571 ± 0.0000  0.7993 ± 0.0057  0.9979 ± 0.0001
Federado + SMPC  0.9840 ± 0.0011  0.8545 ± 0.0015  0.7448 ± 0.0118  0.8571 ± 0.0000  0.7970 ± 0.0068  0.9978 ± 0.0001
Local isolado    0.9821 ± 0.0013  0.8518 ± 0.0007  0.7374 ± 0.0119  0.8571 ± 0.0000  0.7925 ± 0.0069  0.9978 ± 0.0001

>> Δ PAREADO POR SEMENTE (Federado + SMPC − Centralizado):
  ROC-AUC  : -0.0020  (IC95% entre sementes: [-0.0032, -0.0008])
  PR-AUC   : -0.0079  (IC95% entre sementes: [-0.0114, -0.0045])
  Precisão : -0.0039  (IC95% entre sementes: [-0.0176, +0.0097])
  Recall   : +0.0000  (IC95% entre sementes: [+0.0000, +0.0000])
  F1       : -0.0023  (IC95% entre sementes: [-0.0101, +0.0056])
  Acurácia : -0.0000  (IC95% entre sementes: [-0.0001, +0.0001])

>> IC 95% POR BOOTSTRAP DO Δ MÉDIO (300 reamostragens do teste):
  ΔROC-AUC: médio -0.0020 | IC95% [-0.0052, +0.0003] -> não-inferior (margem 0.01)
  ΔPR-AUC: médio -0.0079 | IC95% [-0.0221, +0.0007] -> não-inferioridade NÃO demonstrada (margem 0.01)

>> RESPOSTA À QUESTÃO DE PESQUISA (métrica principal: ROC-AUC):
  O STEP-GAN Federado + SMPC é NÃO-INFERIOR ao baseline centralizado: limite inferior do IC95% -0.0052 > -0.01.
  Ganho da colaboração (Federado + SMPC − Local isolado): +0.0019 ROC-AUC
  SMPC: erro máx. vs FedAvg em claro 8.9e-08

>> TEMPOS (s) E CHECKPOINTS ESCOLHIDOS POR SEMENTE:
         Centralizado  Federado (sequencial)  Federado (paralelo, estimado)  SMPC total Erro máx. SMPC  Bloco escolhido (c
Semente
42              58.64                  61.04                          20.35        1.78        8.9e-08
43              59.22                  61.12                          20.37        1.77        8.9e-08
44              60.08                  60.83                          20.28        2.09        8.9e-08
45              59.95                  59.94                          19.98        2.10        8.9e-08
46              59.40                  61.40                          20.47        1.77        8.9e-08
Resultados por semente salvos em 'resultados_teste6_creditcard.csv'.
```




```
[SUCESSO] Teste 6 (v2) concluído!
```

Nova base: Healthcare Fraud Detection

As células abaixo foram **adicionadas** ao Teste 6, sem alterar as anteriores. Elas repetem o experimento da seção 5 (baseline centralizado × STEP-GAN Federado + SMPC) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — na base `healthcare_fraud_detection.csv`, carregada direto do Google Drive (`/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/`). O rótulo de fraude é a coluna `Is_Fraud`.

1. A primeira célula registra a base (caminho e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base healthcare.
3. A terceira compara os resultados com as outras bases executadas neste notebook.

Engenharia de features da base healthcare (a mesma nos seis testes):

- removidos `Provider_ID` e `Claim_ID` (identificadores, como `nameOrig` / `nameDest` na PaySim) e `Claim_Submission_Date` (carimbo de tempo, como `Time` na Credit Card);
- `claim_amount_log` e `approved_amount_log` (log dos valores, como `amount_log`) e `approved_ratio` = valor aprovado ÷ valor solicitado (análoga ao `errorBalance` da PaySim);
- One-Hot das categóricas (`Patient_Gender`, `Diagnosis_Code`, `Procedure_Code`, `Insurance_Type`, `Provider_Specialty`, `Patient_State`, `Claim_Status`, `Visit_Type`), com ausentes na categoria `Desconhecido`;
- mediana para numéricas ausentes (`Prior_Visits_12m`).

```
# ======================================================================
# --- NOVA BASE: Healthcare Fraud Detection (mesma configuração do teste) ---
# ======================================================================
CAMINHOS['healthcare'] = [
    '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv',
    'base_de_dados/healthcare_fraud_detection.csv',        # Cópia no repositório: executando de dentro de PoC/
    'PoC/base_de_dados/healthcare_fraud_detection.csv',    # Executando da raiz do repositório
]
COLUNAS_CATEGORICAS_HC = ['Patient_Gender', 'Diagnosis_Code', 'Procedure_Code', 'Insurance_Type',
                          'Provider_Specialty', 'Patient_State', 'Claim_Status', 'Visit_Type']


def preparar_healthcare(df):
    # Mesma amostragem das outras bases (todas as fraudes + até 200 normais por fraude);
    # como esta base tem 8,3% de fraude, ela é mantida inteira.
    df = amostrar(df.dropna(subset=['Is_Fraud']), 'Is_Fraud')
    df = df.copy()
    df['Is_Fraud'] = df['Is_Fraud'].astype(int)
    # Identificadores não generalizam (como nameOrig/nameDest); a data é um carimbo de tempo (como Time)
    df = df.drop(columns=['Provider_ID', 'Claim_ID', 'Claim_Submission_Date'], errors='ignore')
    # Log dos valores (como amount_log) e proporção aprovada pela operadora (análoga ao errorBalance da PaySim)
    df['approved_ratio'] = df['Approved_Amount'] / df['Claim_Amount'].replace(0, np.nan)
    df['claim_amount_log'] = np.log1p(df['Claim_Amount'].clip(lower=0))
    df['approved_amount_log'] = np.log1p(df['Approved_Amount'].clip(lower=0))
    df = df.drop(columns=['Claim_Amount', 'Approved_Amount'])
    # Categóricas -> One-Hot (Procedure_Code é código, não quantidade); ausente vira 'Desconhecido'
    for col in COLUNAS_CATEGORICAS_HC:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
    df = pd.get_dummies(df, columns=COLUNAS_CATEGORICAS_HC, drop_first=True, dtype=float)
    # Numéricas ausentes (Prior_Visits_12m) -> mediana
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'Is_Fraud'


PREPARADORES['healthcare'] = preparar_healthcare
DATASETS_HEALTHCARE = ['healthcare']
print("Base healthcare registrada:", CAMINHOS['healthcare'][0])
```

```
Base healthcare registrada: /content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv
```

```
# ======================================================================
# --- 5b. EXPERIMENTO NA BASE HEALTHCARE (mesma configuração da seção 5) ---
# ======================================================================
from sklearn.metrics import roc_curve

METRICA_PRINCIPAL = 'ROC-AUC'       # Métrica do teste de não-inferioridade ('ROC-AUC' ou 'PR-AUC')
MARGEM_NAO_INFERIORIDADE = 0.01     # Perda máxima aceitável do federado em relação ao centralizado
N_BOOTSTRAP = 300                   # Reamostragens do teste para o IC 95% do Δ médio

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CENTRAL, FEDERADO, LOCAL = 'Centralizado', 'Federado + SMPC', 'Local isolado'
CONFIGS = [CENTRAL, FEDERADO, LOCAL]
```

```
CORES = {CENTRAL: '#1f77b4', FEDERADO: '#d62728', LOCAL: '#7f7f7f'}


def treinar_em_blocos(dados, X, y, semente, id_semente_bloco):
    """
    Treino SEM federação com o mesmo procedimento do FL: parte dos mesmos pesos iniciais, treina NUM_RODADAS blocos
    de EPOCAS_LOCAIS épocas e guarda o checkpoint com melhor PR-AUC na validação.
    """
    input_dim = dados['input_dim']
    seed_everything(semente)
    estado = novo_estado_global(input_dim)          # Mesma inicialização de executar_federado()
    historico, melhor, t_total = [], None, 0.0
    for bloco in range(1, NUM_RODADAS + 1):
        sincronizar()
        t0 = time.perf_counter()
        estado = treinar_local(estado, X, y, input_dim, semente * 1000 + bloco * 10 + id_semente_bloco)
        sincronizar()
        t_total += time.perf_counter() - t0
        pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado['D'], input_dim, dados['X_val']))
        historico.append(dict(rodada=bloco, pr_auc_val=pr_auc_val))
        if melhor is None or pr_auc_val > melhor[0]:
            melhor = (pr_auc_val, bloco, copy.deepcopy(estado))
    return melhor[2], melhor[1], pd.DataFrame(historico), t_total


def bootstrap_delta(y, pares_scores, n_boot, semente):
    """IC 95% do Δ médio entre sementes (federado − centralizado), reamostrando o conjunto de teste."""
    rng = np.random.default_rng(semente)
    deltas = {'ROC-AUC': [], 'PR-AUC': []}
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
        if y[idx].min() == y[idx].max():
            continue
        for m, fn in [('ROC-AUC', roc_auc_score), ('PR-AUC', average_precision_score)]:
            deltas[m].append(np.mean([fn(y[idx], f[idx]) - fn(y[idx], c[idx]) for c, f in pares_scores]))
    return {m: np.array(v) for m, v in deltas.items()}


resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_HEALTHCARE:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue
    X_central = np.concatenate([c[0] for c in dados['clientes']])   # União exata dos dados dos clientes
    y_central = np.concatenate([c[1] for c in dados['clientes']])

    linhas, tempos, pares_scores, curvas, exemplo = [], [], [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        # (a) Centralizado
        est_c, bloco_c, hist_c, t_c = treinar_em_blocos(dados, X_central, y_central, semente, id_semente_bloco=9)
        print(f"[Centralizado] treino {t_c:.1f}s | melhor bloco {bloco_c} | PR-AUC val {hist_c['pr_auc_val'].max():.4f}")

        # (b) Federado + SMPC (modelo final)
        est_f, rodada_f, hist_f = executar_federado(dados, semente)
        t_f = hist_f['t_treino'].sum()

        # (c) Local isolado: cada cliente com as mesmas sementes que usa no federado
        avs_locais = []
        for k, (X_k, y_k) in enumerate(dados['clientes']):
            est_k, _, _, _ = treinar_em_blocos(dados, X_k, y_k, semente, id_semente_bloco=k)
            avs_locais.append(avaliar(est_k['D'], dados))
        print(f"[Local isolado] PR-AUC teste por cliente: {[round(a['PR-AUC'], 4) for a in avs_locais]}")

        av = {CENTRAL: avaliar(est_c['D'], dados), FEDERADO: avaliar(est_f['D'], dados),
              LOCAL: {m: np.mean([a[m] for a in avs_locais]) for m in METRICAS}}
        for cfg in CONFIGS:
            linhas.append({'Semente': semente, 'Configuração': cfg, **{m: av[cfg][m] for m in METRICAS}})
            print(f"  {cfg:16s} ROC-AUC {av[cfg]['ROC-AUC']:.4f} | PR-AUC {av[cfg]['PR-AUC']:.4f} | F1 {av[cfg]['F1']:.4f}"
        tempos.append({'Semente': semente, 'Centralizado': t_c, 'Federado (sequencial)': t_f,
                       'Federado (paralelo, estimado)': t_f / NUM_CLIENTES,
                       'SMPC total': hist_f['t_smpc'].sum(), 'Erro máx. SMPC': hist_f['erro_max'].max(),
                       'Bloco escolhido (central)': bloco_c, 'Rodada escolhida (federado)': rodada_f})
        pares_scores.append((av[CENTRAL]['scores'], av[FEDERADO]['scores']))
        curvas.append((hist_c.assign(semente=semente, cfg=CENTRAL), hist_f[['rodada', 'pr_auc_val']].assign(semente=semente
        if exemplo is None:
```

```
            exemplo = {cfg: av[cfg]['scores'] for cfg in [CENTRAL, FEDERADO]}

    df = pd.DataFrame(linhas)
    df_tempos = pd.DataFrame(tempos).set_index('Semente')
    df_curvas = pd.concat([pd.concat(c) for c in curvas], ignore_index=True)

    # ----- Resumo -----
    resumo = df.groupby('Configuração')[METRICAS].agg(['mean', 'std']).reindex(CONFIGS)
    print(f"\n>> MÉTRICAS NO TESTE: média ± desvio-padrão em {len(SEMENTES)} sementes")
    print(pd.DataFrame({m: [f"{resumo.loc[c, (m, 'mean')]:.4f} ± {np.nan_to_num(resumo.loc[c, (m, 'std')]):.4f}"
                            for c in CONFIGS] for m in METRICAS}, index=CONFIGS).to_string())

    piv = df.pivot(index='Semente', columns='Configuração')
    deltas = {m: (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).values for m in METRICAS}
    print(f"\n>> Δ PAREADO POR SEMENTE ({FEDERADO} − {CENTRAL}):")
    for m in METRICAS:
        d = deltas[m]
        meia = stats.t.ppf(0.975, len(d) - 1) * d.std(ddof=1) / np.sqrt(len(d)) if len(d) > 1 else 0.0
        print(f"  {m:9s}: {d.mean():+.4f}  (IC95% entre sementes: [{d.mean() - meia:+.4f}, {d.mean() + meia:+.4f}])")

    boot = bootstrap_delta(dados['y_test'], pares_scores, N_BOOTSTRAP, SEMENTES[0])
    ic = {m: np.percentile(v, [2.5, 97.5]) for m, v in boot.items()}
    print(f"\n>> IC 95% POR BOOTSTRAP DO Δ MÉDIO ({N_BOOTSTRAP} reamostragens do teste):")
    for m in ['ROC-AUC', 'PR-AUC']:
        veredito = 'não-inferior' if ic[m][0] > -MARGEM_NAO_INFERIORIDADE else 'não-inferioridade NÃO demonstrada'
        print(f"  Δ{m}: médio {deltas[m].mean():+.4f} | IC95% [{ic[m][0]:+.4f}, {ic[m][1]:+.4f}] -> {veredito} "
              f"(margem {MARGEM_NAO_INFERIORIDADE})")

    nao_inferior = ic[METRICA_PRINCIPAL][0] > -MARGEM_NAO_INFERIORIDADE
    print(f"\n>> RESPOSTA À QUESTÃO DE PESQUISA (métrica principal: {METRICA_PRINCIPAL}):")
    if nao_inferior:
        print(f"  O STEP-GAN Federado + SMPC é NÃO-INFERIOR ao baseline centralizado: limite inferior do IC95% "
              f"{ic[METRICA_PRINCIPAL][0]:+.4f} > {-MARGEM_NAO_INFERIORIDADE}.")
    else:
        print(f"  NÃO foi possível demonstrar não-inferioridade: limite inferior do IC95% "
              f"{ic[METRICA_PRINCIPAL][0]:+.4f} <= {-MARGEM_NAO_INFERIORIDADE}.")
    ganho = (piv[(METRICA_PRINCIPAL, FEDERADO)] - piv[(METRICA_PRINCIPAL, LOCAL)]).mean()
    print(f"  Ganho da colaboração ({FEDERADO} − {LOCAL}): {ganho:+.4f} {METRICA_PRINCIPAL}")
    print(f"  SMPC: erro máx. vs FedAvg em claro {df_tempos['Erro máx. SMPC'].max():.1e}")

    print("\n>> TEMPOS (s) E CHECKPOINTS ESCOLHIDOS POR SEMENTE:")
    print(df_tempos.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}"}, float_format=lambda x: f"{x:.2f}"))

    arquivo = f'resultados_teste6_{nome_base}.csv'
    df.to_csv(arquivo, index=False)
    print(f"Resultados por semente salvos em '{arquivo}'.")
    resultados_gerais[nome_base] = dict(metricas=df, tempos=df_tempos, ic_bootstrap=ic, nao_inferior=nao_inferior)

    # ----- Visualizações -----
    fig, axes = plt.subplots(2, 3, figsize=(22, 11))
    fig.suptitle(f"Teste 6 (v2): Centralizado × Federado + SMPC (STEP-GAN v5, {nome_base}, {len(SEMENTES)} sementes)",
                 fontsize=14, fontweight='bold')

    # 1. Métricas
    ax = axes[0, 0]
    largura = 0.8 / len(CONFIGS)
    x = np.arange(len(METRICAS))
    for i, cfg in enumerate(CONFIGS):
        ax.bar(x + (i - (len(CONFIGS) - 1) / 2) * largura, [resumo.loc[cfg, (m, 'mean')] for m in METRICAS], largura,
               yerr=[np.nan_to_num(resumo.loc[cfg, (m, 'std')]) for m in METRICAS], capsize=3,
               color=CORES[cfg], label=cfg)
    ax.set_xticks(x)
    ax.set_xticklabels(METRICAS)
    ax.set_ylim(0, 1.2)
    ax.set_title('Métricas no teste (média ± desvio)', fontweight='bold')
    ax.legend(loc='upper center', ncol=3)

    # 2. Curvas ROC (primeira semente)
    ax = axes[0, 1]
    for cfg in [CENTRAL, FEDERADO]:
        fpr, tpr, _ = roc_curve(dados['y_test'], exemplo[cfg])
        ax.plot(fpr, tpr, '-' if cfg == CENTRAL else '--', color=CORES[cfg],
                label=f"{cfg} (AUC={roc_auc_score(dados['y_test'], exemplo[cfg]):.4f})")
    ax.plot([0, 1], [0, 1], ':', color='gray')
    ax.set_title(f'Curva ROC no teste (semente {SEMENTES[0]})', fontweight='bold')
    ax.set_xlabel('FPR')
    ax.set_ylabel('TPR')
    ax.legend(loc='lower right')

    # 3. Convergência: PR-AUC na validação por bloco/rodada
    ax = axes[0, 2]
    for cfg in [CENTRAL, FEDERADO]:
```

```
            sub = df_curvas[df_curvas['cfg'] == cfg]
            for _, g in sub.groupby('semente'):
                ax.plot(g['rodada'], g['pr_auc_val'], color=CORES[cfg], alpha=0.25)
            media = sub.groupby('rodada')['pr_auc_val'].mean()
            ax.plot(media.index, media.values, color=CORES[cfg], lw=2.5, marker='o', label=f'{cfg} (média)')
        ax.set_xlabel(f'Rodada / bloco de {EPOCAS_LOCAIS} épocas')
        ax.set_title('PR-AUC na validação', fontweight='bold')
        ax.legend()

        # 4. Δ por semente e IC de não-inferioridade
        ax = axes[1, 0]
        for i, m in enumerate(['ROC-AUC', 'PR-AUC', 'F1']):
            ax.scatter(np.full(len(deltas[m]), i), deltas[m], color=CORES[FEDERADO], zorder=3,
                       label='Δ por semente' if i == 0 else None)
            ax.plot([i - 0.2, i + 0.2], [deltas[m].mean()] * 2, color='black', lw=2, label='Δ médio' if i == 0 else None)
        for i, m in enumerate(['ROC-AUC', 'PR-AUC']):
            ax.errorbar(i + 0.3, np.mean(ic[m]), yerr=[[np.mean(ic[m]) - ic[m][0]], [ic[m][1] - np.mean(ic[m])]],
                        fmt='none', ecolor='#2ca02c', capsize=6, lw=2, label='IC95% bootstrap' if i == 0 else None)
        ax.axhline(0, color='gray', lw=1)
        ax.axhline(-MARGEM_NAO_INFERIORIDADE, color='#d62728', ls='--',
                   label=f'Margem de não-inferioridade (−{MARGEM_NAO_INFERIORIDADE})')
        ax.set_xticks(range(3))
        ax.set_xticklabels(['ΔROC-AUC', 'ΔPR-AUC', 'ΔF1'])
        ax.set_title(f'{FEDERADO} − {CENTRAL}', fontweight='bold')
        ax.legend(loc='best', fontsize=9)

        # 5. Tempo de treino
        ax = axes[1, 1]
        media_t = df_tempos[['Centralizado', 'Federado (sequencial)', 'Federado (paralelo, estimado)']].mean()
        ax.bar(media_t.index, media_t.values, color=['#1f77b4', '#ff9896', '#d62728'])
        for i, v in enumerate(media_t.values):
            ax.text(i, v, f"{v:.1f}s", ha='center', va='bottom')
        ax.set_title(f"Tempo de treino (SMPC total: {df_tempos['SMPC total'].mean():.2f}s)", fontweight='bold')
        ax.set_ylabel('segundos')

        axes[1, 2].axis('off')
        plt.tight_layout()
        plt.show()

    print("\n[SUCESSO] Teste 6 (v2) concluído na base healthcare!")
```

```
================================================================================
 BASE: HEALTHCARE | 5 sementes | 5 x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv'...
[DADOS] Features: 48 | Treino: 6000 (fraudes: 497) | Validação: 2000 (fraudes: 166) | Teste: 2000 (fraudes: 166)

--- Semente 42 ---
[Centralizado] treino 6.3s | melhor bloco 5 | PR-AUC val 0.9850
  Rodada 1/5 | PR-AUC val 0.7073 | treino 1.2s | SMPC 376 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8326 | treino 1.2s | SMPC 375 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.9088 | treino 1.2s | SMPC 381 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9436 | treino 1.5s | SMPC 542 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9620 | treino 1.7s | SMPC 416 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9633), np.float64(0.9645), np.float64(0.9583)]
  Centralizado     ROC-AUC 0.9990 | PR-AUC 0.9910 | F1 0.8367
  Federado + SMPC  ROC-AUC 0.9965 | PR-AUC 0.9727 | F1 0.8122
  Local isolado    ROC-AUC 0.9951 | PR-AUC 0.9620 | F1 0.8013

--- Semente 43 ---
[Centralizado] treino 6.9s | melhor bloco 5 | PR-AUC val 0.9838
  Rodada 1/5 | PR-AUC val 0.6179 | treino 1.2s | SMPC 371 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.7657 | treino 1.2s | SMPC 380 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8848 | treino 1.2s | SMPC 367 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9351 | treino 1.2s | SMPC 378 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9568 | treino 1.2s | SMPC 372 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9709), np.float64(0.9808), np.float64(0.9691)]
  Centralizado     ROC-AUC 0.9994 | PR-AUC 0.9943 | F1 0.8342
  Federado + SMPC  ROC-AUC 0.9981 | PR-AUC 0.9830 | F1 0.8180
  Local isolado    ROC-AUC 0.9968 | PR-AUC 0.9736 | F1 0.8111

--- Semente 44 ---
[Centralizado] treino 5.9s | melhor bloco 5 | PR-AUC val 0.9852
  Rodada 1/5 | PR-AUC val 0.7296 | treino 1.2s | SMPC 376 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8501 | treino 1.5s | SMPC 533 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.9050 | treino 1.6s | SMPC 370 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9349 | treino 1.2s | SMPC 372 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9549 | treino 1.2s | SMPC 370 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.9619), np.float64(0.9683), np.float64(0.9533)]
  Centralizado     ROC-AUC 0.9994 | PR-AUC 0.9940 | F1 0.8405
  Federado + SMPC  ROC-AUC 0.9966 | PR-AUC 0.9727 | F1 0.8030
  Local isolado    ROC-AUC 0.9950 | PR-AUC 0.9611 | F1 0.8096

--- Semente 45 ---
[Centralizado] treino 6.8s | melhor bloco 5 | PR-AUC val 0.9836
  Rodada 1/5 | PR-AUC val 0.6624 | treino 1.2s | SMPC 380 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.7910 | treino 1.2s | SMPC 372 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8813 | treino 1.2s | SMPC 385 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9293 | treino 1.2s | SMPC 374 ms (erro máx. vs claro 8.9e-08)
```

```
# ======================================================================
# --- COMPARATIVO ENTRE AS BASES (Teste 6: Centralizado × Federado + SMPC) ---
# ======================================================================
import contextlib
import io

NOMES_BASES = {'creditcard': 'Credit Card', 'paysim': 'PaySim', 'healthcare': 'Healthcare'}
# Cor fixa por base (paleta categórica validada para daltonismo): a mesma base tem sempre a mesma cor
CORES_BASES = {'creditcard': '#2a78d6', 'healthcare': '#eb6834', 'paysim': '#1baf7a'}
bases = [b for b in NOMES_BASES if b in globals().get('resultados_gerais', {})]


def perfil_da_base(nome):
    """Tamanho e taxa de fraude do conjunto de teste (a base é recarregada só para essas estatísticas)."""
    with contextlib.redirect_stdout(io.StringIO()):
        d = carregar_dados(nome)
    return {'Transações (teste)': len(d['y_test']), 'Taxa de fraude (teste)': float(np.mean(d['y_test'])),
            'Features': int(d['input_dim'])}

METRICAS_T6 = ['ROC-AUC', 'PR-AUC', 'F1']

if not bases:
    print("Nenhum resultado disponível: execute a seção 5 e/ou a base healthcare antes do comparativo.")
else:
    perfis = {b: perfil_da_base(b) for b in bases}
    linhas_metricas, linhas_delta = [], []
    for b in bases:
        metricas_b, ic_b = resultados_gerais[b]['metricas'], resultados_gerais[b]['ic_bootstrap']
        for cfg in CONFIGS:
            sub = metricas_b[metricas_b['Configuração'] == cfg]
            taxa = perfis[b]['Taxa de fraude (teste)']
            linhas_metricas.append({'Base': NOMES_BASES[b], 'Configuração': cfg,
                                    **{m: f"{sub[m].mean():.4f} ± {sub[m].std(ddof=1):.4f}" for m in METRICAS_T6},
                                    'PR-AUC normalizado': f"{((sub['PR-AUC'] - taxa) / (1 - taxa)).mean():.4f}"})
        piv = metricas_b.pivot(index='Semente', columns='Configuração')
        linha = {'Base': NOMES_BASES[b], 'Taxa de fraude (teste)': f"{perfis[b]['Taxa de fraude (teste)']:.2%}"}
        for m in ['ROC-AUC', 'PR-AUC']:
```

```
            delta = (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).mean()
            linha[f'Δ{m} (fed − central)'] = f"{delta:+.4f}"
            linha[f'IC95% Δ{m}'] = f"[{ic_b[m][0]:+.4f}; {ic_b[m][1]:+.4f}]"
            linha[f'Não-inferior em {m}?'] = 'sim' if ic_b[m][0] > -MARGEM_NAO_INFERIORIDADE else 'não demonstrado'
        linha['Ganho da colaboração (fed − local, ROC-AUC)'] = f"{(piv[('ROC-AUC', FEDERADO)] - piv[('ROC-AUC', LOCAL)]).me
        linhas_delta.append(linha)

    print(f">> MÉTRICAS NO TESTE POR BASE E CONFIGURAÇÃO ({len(SEMENTES)} sementes; média ± desvio-padrão)")
    print(pd.DataFrame(linhas_metricas).set_index(['Base', 'Configuração']).to_string())
    print(f"\n>> NÃO-INFERIORIDADE DO FEDERADO + SMPC POR BASE (margem {MARGEM_NAO_INFERIORIDADE}; "
          f"métrica principal: {METRICA_PRINCIPAL})")
    print(pd.DataFrame(linhas_delta).set_index('Base').T.to_string())
    print("\nObservação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. "
          "O PR-AUC depende dela (um classificador aleatório tem PR-AUC igual à taxa de fraude); por isso a comparação "
          "do PR-AUC entre bases deve usar o PR-AUC normalizado = (PR-AUC − taxa) / (1 − taxa): 0 = aleatório, 1 = perfeito

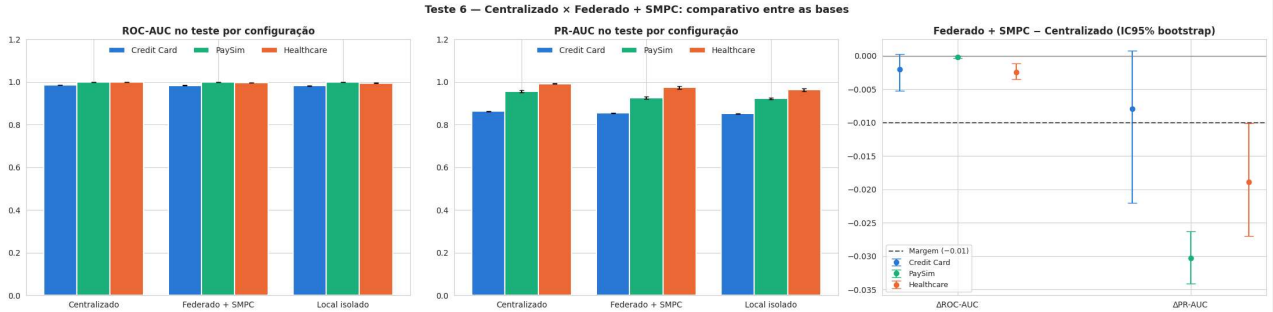
    fig, axes = plt.subplots(1, 3, figsize=(22, 5.5))
    largura = 0.8 / len(bases)
    x = np.arange(len(CONFIGS))
    for eixo, m in zip(axes[:2], ['ROC-AUC', 'PR-AUC']):
        for i, b in enumerate(bases):
            metricas_b = resultados_gerais[b]['metricas']
            medias = [metricas_b.loc[metricas_b['Configuração'] == c, m].mean() for c in CONFIGS]
            desvios = [metricas_b.loc[metricas_b['Configuração'] == c, m].std(ddof=1) for c in CONFIGS]
            eixo.bar(x + (i - (len(bases) - 1) / 2) * largura, medias, largura, yerr=desvios, capsize=3,
                     color=CORES_BASES[b], label=NOMES_BASES[b], edgecolor='white', linewidth=1.5)
        eixo.set_xticks(x)
        eixo.set_xticklabels(CONFIGS)
        eixo.set_ylim(0, 1.2)
        eixo.set_title(f'{m} no teste por configuração', fontweight='bold')
        eixo.legend(loc='upper center', ncol=len(bases), frameon=False)

    for i, b in enumerate(bases):
        piv = resultados_gerais[b]['metricas'].pivot(index='Semente', columns='Configuração')
        ic_b = resultados_gerais[b]['ic_bootstrap']
        for j, m in enumerate(['ROC-AUC', 'PR-AUC']):
            centro = (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).mean()
            posicao = j + (i - (len(bases) - 1) / 2) * 0.25
            axes[2].errorbar(posicao, centro, yerr=[[centro - ic_b[m][0]], [ic_b[m][1] - centro]], fmt='o', capsize=6,
                             color=CORES_BASES[b], label=NOMES_BASES[b] if j == 0 else None)
    axes[2].axhline(0, color='gray', lw=1)
    axes[2].axhline(-MARGEM_NAO_INFERIORIDADE, color='#52514e', ls='--', label=f'Margem (−{MARGEM_NAO_INFERIORIDADE})')
    axes[2].set_xticks([0, 1])
    axes[2].set_xticklabels(['ΔROC-AUC', 'ΔPR-AUC'])
    axes[2].set_title('Federado + SMPC − Centralizado (IC95% bootstrap)', fontweight='bold')
    axes[2].legend(fontsize=9)
    plt.suptitle('Teste 6 — Centralizado × Federado + SMPC: comparativo entre as bases', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
>> MÉTRICAS NO TESTE POR BASE E CONFIGURAÇÃO (5 sementes; média ± desvio-padrão)
                                ROC-AUC           PR-AUC              F1 PR-AUC normalizado
Base        Configuração
Credit Card Centralizado     0.9860 ± 0.0006  0.8624 ± 0.0019  0.7993 ± 0.0057             0.8617
            Federado + SMPC  0.9840 ± 0.0011  0.8545 ± 0.0015  0.7970 ± 0.0068             0.8538
            Local isolado    0.9821 ± 0.0013  0.8518 ± 0.0007  0.7925 ± 0.0069             0.8511
PaySim      Centralizado     0.9994 ± 0.0002  0.9562 ± 0.0043  0.8070 ± 0.0050             0.9559
            Federado + SMPC  0.9992 ± 0.0000  0.9258 ± 0.0049  0.7792 ± 0.0038             0.9255
            Local isolado    0.9991 ± 0.0000  0.9231 ± 0.0040  0.7789 ± 0.0036             0.9227
Healthcare  Centralizado     0.9992 ± 0.0002  0.9927 ± 0.0016  0.8393 ± 0.0045             0.9920
            Federado + SMPC  0.9968 ± 0.0010  0.9738 ± 0.0063  0.8196 ± 0.0140             0.9714
            Local isolado    0.9953 ± 0.0010  0.9630 ± 0.0068  0.8146 ± 0.0133             0.9597

>> NÃO-INFERIORIDADE DO FEDERADO + SMPC POR BASE (margem 0.01; métrica principal: ROC-AUC)
Base                                              Credit Card              PaySim          Healthcare
Taxa de fraude (teste)                                  0.50%               0.50%               8.30%
ΔROC-AUC (fed − central)                              -0.0020             -0.0002             -0.0024
IC95% ΔROC-AUC                              [-0.0052; +0.0003]  [-0.0004; -0.0000]  [-0.0035; -0.0011]
Não-inferior em ROC-AUC?                                 sim                 sim                 sim
ΔPR-AUC (fed − central)                               -0.0079             -0.0303             -0.0189
IC95% ΔPR-AUC                               [-0.0221; +0.0007]  [-0.0341; -0.0263]  [-0.0270; -0.0101]
Não-inferior em PR-AUC?                       não demonstrado     não demonstrado     não demonstrado
Ganho da colaboração (fed − local, ROC-AUC)           +0.0019             +0.0001             +0.0015

Observação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. O PR-AUC depende dela
```



Novas bases: Bank Account Fraud (NeurIPS 2022), Insurance Claims e Fraud Oracle

As células abaixo foram **adicionadas** ao Teste 6, sem alterar as anteriores. Elas repetem o experimento da seção 5 (baseline centralizado × STEP-GAN Federado + SMPC) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — em mais três bases.

Origem das bases:

- **Bank Account Fraud Dataset Suite (NeurIPS 2022)** — só o arquivo `Base.csv` (213 MB, 1 milhão de solicitações de abertura de conta, ~1,1% de fraude), baixado com `kagglehub` (já instalado no Colab; localmente, `pip install kagglehub`): `kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")`. Sem acesso ao Kaggle, é usada a cópia do repositório, `PoC/base_de_dados/Base.csv.zip`.
- **Insurance Claims** — `insurance_claims.csv` (1.000 sinistros de seguro de automóvel, 24,7% de fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/insurance_claims.csv`: a célula usa a cópia local quando o notebook roda a partir do repositório e, no Colab, lê o arquivo direto do GitHub.
- **Fraud Oracle** — `fraud_oracle.csv` (15.420 sinistros de seguro de veículos, 6,0% de fraude; rótulo `FraudFound_P`: 1 = fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/fraud_oracle.csv`, lida da mesma forma que a Insurance Claims.

1. A primeira célula registra as três bases (origem e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base BAF (seção 5c).
3. A terceira executa o experimento na base Insurance Claims (seção 5d).
4. A quarta executa o experimento na base Fraud Oracle (seção 5e).
5. A quinta compara os resultados de todas as bases executadas neste notebook (Credit Card, Healthcare, BAF, Insurance Claims e Fraud Oracle).

Tamanho das bases: a amostragem é a mesma das outras bases (todas as fraudes + até 200 normais por fraude), e as três entram inteiras. A BAF (1 milhão de linhas, no porte da PaySim amostrada) deixa o experimento bem mais demorado que na Credit Card; para uma execução exploratória mais rápida, `AMOSTRA_MAXIMA_NOVAS_BASES` (célula de registro) limita cada base a uma amostra estratificada, e o padrão `None` mantém a configuração do teste. A Insurance Claims é pequena (1.000 linhas, ~200 de treino por cliente): os resultados dela oscilam mais entre sementes e o IC fica mais largo. A Fraud Oracle (15.420 linhas) roda em poucos minutos.

Engenharia de features (a mesma nos seis testes):

- **BAF** (rótulo `fraud_bool`): removidos `month` (mês da solicitação, carimbo de tempo como `Time` na Credit Card) e `device_fraud_count` (constante na `Base.csv`); nas seis colunas em que valor negativo significa ausente (`prev_address_months_count`, `current_address_months_count`, `intended_balcon_amount`, `bank_months_count`, `session_length_in_minutes`, `device_distinct_emails_8w`), criado um indicador `_ausente` e o valor ausente vai para a mediana; log de `intended_balcon_amount`, `days_since_request`, `zip_count_4w` e `proposed_credit_limit`; One-Hot de `payment_type`, `employment_status`, `housing_status`, `source` e `device_os`.
- **Insurance Claims** (rótulo `fraud_reported`, Y/N → 1/0): removidos `policy_number`, `insured_zip` e `incident_location` (identificadores, como `nameOrig`/`nameDest` na PaySim), `_c39` (coluna vazia) e `auto_model` (39 modelos para 1.000 linhas; `auto_make` é mantida); as datas `policy_bind_date` e `incident_date` viram `dias_apolice_ate_incidente`; log de `total_claim_amount`, `injury_claim`, `property_claim` e `vehicle_claim`; One-Hot das 16 categóricas, com `?` na categoria `Desconhecido` e `authorities_contacted` ausente (texto `None` no CSV) como `Nenhuma`.
- **Fraud Oracle** (rótulo `FraudFound_P`): removidos `PolicyNumber` e `RepNumber` (identificadores), `Year` (carimbo de tempo, como `Time`) e `PolicyType` (repete `VehicleCategory` + `BasePolicy`); `Month` e `MonthClaimed` viram `meses_ate_sinistro`; as nove colunas de faixas em texto (`VehiclePrice`, `Days_Policy_Accident`, `Days_Policy_Claim`, `PastNumberOfClaims`, `AgeOfVehicle`, `AgeOfPolicyHolder`, `NumberOfSuppliments`, `AddressChange_Claim`, `NumberOfCars`) viram números em ordem crescente; `Age` = 0 (inválido) vai para a mediana; One-Hot das 12 categóricas, com `0` em `DayOfWeekClaimed` como `Desconhecido`.

```
# ======================================================================
# --- NOVAS BASES: Bank Account Fraud (Base.csv), Insurance Claims e Fraud Oracle (mesma configuração do teste) ---
# ======================================================================
AMOSTRA_MAXIMA_NOVAS_BASES = None   # None = base inteira (configuração do teste); ex.: 300_000 para
                                    # uma execução exploratória mais rápida (amostra estratificada)


def baixar_base_baf():
    """Baixa só o arquivo Base.csv com kagglehub (None se o Kaggle não estiver acessível)."""
    try:
        import kagglehub
        # Download latest version — apenas o arquivo Base.csv (213 MB)
        path = kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")
        print("Path to dataset files:", path)
        return path
    except Exception as erro:
        print(f"Download pelo kagglehub indisponível ({type(erro).__name__}): usando a cópia do repositório.")
        return None


CAMINHOS['baf'] = [c for c in [
    baixar_base_baf(),
    'base_de_dados/Base.csv.zip',      # Cópia no repositório (zip lido direto pelo pandas): de dentro de PoC/
    'PoC/base_de_dados/Base.csv.zip',  # Executando da raiz do repositório
    'base_de_dados/Base.csv',
    'PoC/base_de_dados/Base.csv',
] if c]
# Insurance Claims: arquivo no repositório GitHub do projeto (PoC/base_de_dados/insurance_claims.csv)
CAMINHOS['insurance'] = [
    'base_de_dados/insurance_claims.csv',        # Executando de dentro de PoC/
    'PoC/base_de_dados/insurance_claims.csv',    # Executando da raiz do repositório
]
if not any(os.path.exists(c) for c in CAMINHOS['insurance']):
    # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
    import urllib.request
    try:
        urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/i
        CAMINHOS['insurance'].append('insurance_claims.csv')
        print("insurance_claims.csv baixado do repositório GitHub.")
    except Exception as erro:
        print(f"Não foi possível baixar insurance_claims.csv do GitHub ({type(erro).__name__}).")
# Fraud Oracle: arquivo no repositório GitHub do projeto (PoC/base_de_dados/fraud_oracle.csv)
CAMINHOS['oracle'] = [
    'base_de_dados/fraud_oracle.csv',        # Executando de dentro de PoC/
    'PoC/base_de_dados/fraud_oracle.csv',    # Executando da raiz do repositório
]
if not any(os.path.exists(c) for c in CAMINHOS['oracle']):
    # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
    import urllib.request
    try:
        urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/f
        CAMINHOS['oracle'].append('fraud_oracle.csv')
        print("fraud_oracle.csv baixado do repositório GitHub.")
    except Exception as erro:
        print(f"Não foi possível baixar fraud_oracle.csv do GitHub ({type(erro).__name__}).")
```

```
def reduzir(df, alvo):
    """Mesma amostragem das outras bases (todas as fraudes + até 200 normais por fraude) e, opcionalmente,
    uma amostra estratificada de AMOSTRA_MAXIMA_NOVAS_BASES linhas."""
    df = amostrar(df.dropna(subset=[alvo]), alvo)
    if AMOSTRA_MAXIMA_NOVAS_BASES and len(df) > AMOSTRA_MAXIMA_NOVAS_BASES:
        print(f"Amostra estratificada de {AMOSTRA_MAXIMA_NOVAS_BASES:,} linhas (preserva a taxa de fraude)...")
        df, _ = train_test_split(df, train_size=AMOSTRA_MAXIMA_NOVAS_BASES, stratify=df[alvo], random_state=42)
    return df.copy()


def preparar_baf(df):
    df = reduzir(df, 'fraud_bool')
    df['fraud_bool'] = df['fraud_bool'].astype(int)
    # month: carimbo de tempo (como Time); device_fraud_count: constante (zero) na Base.csv
    df = df.drop(columns=['month', 'device_fraud_count'], errors='ignore')
    # Valor negativo nestas colunas codifica "ausente" (documentação do dataset): indicador + mediana
    for col in ['prev_address_months_count', 'current_address_months_count', 'intended_balcon_amount',
                'bank_months_count', 'session_length_in_minutes', 'device_distinct_emails_8w']:
        df[col + '_ausente'] = (df[col] < 0).astype(float)
        df[col] = df[col].where(df[col] >= 0)
    # Contagens e valores com cauda longa -> log (como amount_log nas outras bases)
    for col in ['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df = df.drop(columns=['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit'])
    categoricas = ['payment_type', 'employment_status', 'housing_status', 'source', 'device_os']
    for col in categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
    df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=float)
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'fraud_bool'


def preparar_insurance(df):
    df = df.copy()
    df['fraud_reported'] = df['fraud_reported'].astype(str).str.strip().str.upper().map({'Y': 1, 'N': 0})
    df = reduzir(df, 'fraud_reported')
    df['fraud_reported'] = df['fraud_reported'].astype(int)
    # Datas -> um único intervalo (dias entre a contratação da apólice e o incidente)
    df['dias_apolice_ate_incidente'] = (pd.to_datetime(df['incident_date'], errors='coerce')
                                        - pd.to_datetime(df['policy_bind_date'], errors='coerce')).dt.days
    # Identificadores (como nameOrig/nameDest), coluna vazia, auto_model (39 modelos em 1.000 linhas) e as datas
    df = df.drop(columns=['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model',
                          'policy_bind_date', 'incident_date'], errors='ignore')
    # Valores do sinistro com cauda longa -> log (como amount_log nas outras bases)
    for col in ['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df = df.drop(columns=['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim'])
    # '?' = desconhecido; authorities_contacted 'None' (lido como ausente) = nenhuma autoridade acionada
    df['authorities_contacted'] = df['authorities_contacted'].fillna('Nenhuma')
    categoricas = ['policy_state', 'policy_csl', 'insured_sex', 'insured_education_level', 'insured_occupation',
                   'insured_hobbies', 'insured_relationship', 'incident_type', 'collision_type', 'incident_severity',
                   'authorities_contacted', 'incident_state', 'incident_city', 'property_damage',
                   'police_report_available', 'auto_make']
    for col in categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('?', 'Desconhecido')
    df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=float)
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'fraud_reported'


def preparar_oracle(df):
    df = reduzir(df, 'FraudFound_P')
    df['FraudFound_P'] = df['FraudFound_P'].astype(int)
    # PolicyNumber (sequencial) e RepNumber (representante) são identificadores, como nameOrig/nameDest na PaySim;
    # Year é carimbo de tempo, como 'Time' na base Credit Card; PolicyType repete VehicleCategory + BasePolicy.
    # Month e MonthClaimed viram um único intervalo: meses entre o acidente e a abertura do sinistro.
    MESES = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 1
    df['meses_ate_sinistro'] = (df['MonthClaimed'].map(MESES) - df['Month'].map(MESES)) % 12
    colunas_remover = ['PolicyNumber', 'RepNumber', 'Year', 'PolicyType', 'Month', 'MonthClaimed']
    df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
    # Faixas de texto -> números em ordem crescente (preserva a ordem e evita dezenas de colunas One-Hot)
    FAIXAS_ORDINAIS = {
        'VehiclePrice': ['less than 20000', '20000 to 29000', '30000 to 39000', '40000 to 59000', '60000 to 69000', 'more t
        'Days_Policy_Accident': ['none', '1 to 7', '8 to 15', '15 to 30', 'more than 30'],
        'Days_Policy_Claim': ['none', '8 to 15', '15 to 30', 'more than 30'],
        'PastNumberOfClaims': ['none', '1', '2 to 4', 'more than 4'],
```

```
        'AgeOfVehicle': ['new', '2 years', '3 years', '4 years', '5 years', '6 years', '7 years', 'more than 7'],
        'AgeOfPolicyHolder': ['16 to 17', '18 to 20', '21 to 25', '26 to 30', '31 to 35', '36 to 40', '41 to 50', '51 to 65
        'NumberOfSuppliments': ['none', '1 to 2', '3 to 5', 'more than 5'],
        'AddressChange_Claim': ['under 6 months', '1 year', '2 to 3 years', '4 to 8 years', 'no change'],
        'NumberOfCars': ['1 vehicle', '2 vehicles', '3 to 4', '5 to 8', 'more than 8'],
    }
    for col, ordem in FAIXAS_ORDINAIS.items():
        df[col] = df[col].astype(str).str.strip().map({faixa: i for i, faixa in enumerate(ordem)})
    # Age = 0 é um valor inválido na base (o motorista mais novo está na faixa 16 a 17 anos): tratado como ausente
    df['Age'] = df['Age'].where(df['Age'] > 0)

    # Categóricas -> One-Hot ('0' em DayOfWeekClaimed é valor desconhecido)
    colunas_categoricas = ['DayOfWeek', 'DayOfWeekClaimed', 'Make', 'AccidentArea', 'Sex', 'MaritalStatus', 'Fault',
                           'VehicleCategory', 'PoliceReportFiled', 'WitnessPresent', 'AgentType', 'BasePolicy']
    for col in colunas_categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('0', 'Desconhecido')
    df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

    # Numéricas ausentes (faixas fora da lista, Age = 0, mês desconhecido) -> mediana
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'FraudFound_P'


PREPARADORES['baf'] = preparar_baf
PREPARADORES['insurance'] = preparar_insurance
PREPARADORES['oracle'] = preparar_oracle
DATASETS_BAF = ['baf']
DATASETS_INSURANCE = ['insurance']
DATASETS_ORACLE = ['oracle']
for nome_base in ['baf', 'insurance', 'oracle']:
    print(f"Base {nome_base} registrada; caminhos procurados: {CAMINHOS[nome_base]}")
```

```
Using Colab cache for faster access to the 'bank-account-fraud-dataset-neurips-2022' dataset.
Path to dataset files: /kaggle/input/bank-account-fraud-dataset-neurips-2022/Base.csv
insurance_claims.csv baixado do repositório GitHub.
fraud_oracle.csv baixado do repositório GitHub.
Base baf registrada; caminhos procurados: ['/kaggle/input/bank-account-fraud-dataset-neurips-2022/Base.csv', 'base_de_dados/
Base insurance registrada; caminhos procurados: ['base_de_dados/insurance_claims.csv', 'PoC/base_de_dados/insurance_claims.c
Base oracle registrada; caminhos procurados: ['base_de_dados/fraud_oracle.csv', 'PoC/base_de_dados/fraud_oracle.csv', 'fraud
```

```
# ======================================================================
# --- 5c. EXPERIMENTO NA BASE BAF (mesma configuração da seção 5) ---
# ======================================================================
from sklearn.metrics import roc_curve

METRICA_PRINCIPAL = 'ROC-AUC'       # Métrica do teste de não-inferioridade ('ROC-AUC' ou 'PR-AUC')
MARGEM_NAO_INFERIORIDADE = 0.01     # Perda máxima aceitável do federado em relação ao centralizado
N_BOOTSTRAP = 300                   # Reamostragens do teste para o IC 95% do Δ médio

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CENTRAL, FEDERADO, LOCAL = 'Centralizado', 'Federado + SMPC', 'Local isolado'
CONFIGS = [CENTRAL, FEDERADO, LOCAL]
CORES = {CENTRAL: '#1f77b4', FEDERADO: '#d62728', LOCAL: '#7f7f7f'}


def treinar_em_blocos(dados, X, y, semente, id_semente_bloco):
    """
    Treino SEM federação com o mesmo procedimento do FL: parte dos mesmos pesos iniciais, treina NUM_RODADAS blocos
    de EPOCAS_LOCAIS épocas e guarda o checkpoint com melhor PR-AUC na validação.
    """
    input_dim = dados['input_dim']
    seed_everything(semente)
    estado = novo_estado_global(input_dim)          # Mesma inicialização de executar_federado()
    historico, melhor, t_total = [], None, 0.0
    for bloco in range(1, NUM_RODADAS + 1):
        sincronizar()
        t0 = time.perf_counter()
        estado = treinar_local(estado, X, y, input_dim, semente * 1000 + bloco * 10 + id_semente_bloco)
        sincronizar()
        t_total += time.perf_counter() - t0
        pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado['D'], input_dim, dados['X_val']))
        historico.append(dict(rodada=bloco, pr_auc_val=pr_auc_val))
        if melhor is None or pr_auc_val > melhor[0]:
            melhor = (pr_auc_val, bloco, copy.deepcopy(estado))
    return melhor[2], melhor[1], pd.DataFrame(historico), t_total


def bootstrap_delta(y, pares_scores, n_boot, semente):
    """IC 95% do Δ médio entre sementes (federado − centralizado), reamostrando o conjunto de teste."""
```

```
    rng = np.random.default_rng(semente)
    deltas = {'ROC-AUC': [], 'PR-AUC': []}
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
        if y[idx].min() == y[idx].max():
            continue
        for m, fn in [('ROC-AUC', roc_auc_score), ('PR-AUC', average_precision_score)]:
            deltas[m].append(np.mean([fn(y[idx], f[idx]) - fn(y[idx], c[idx]) for c, f in pares_scores]))
    return {m: np.array(v) for m, v in deltas.items()}


resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_BAF:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue
    X_central = np.concatenate([c[0] for c in dados['clientes']])   # União exata dos dados dos clientes
    y_central = np.concatenate([c[1] for c in dados['clientes']])

    linhas, tempos, pares_scores, curvas, exemplo = [], [], [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        # (a) Centralizado
        est_c, bloco_c, hist_c, t_c = treinar_em_blocos(dados, X_central, y_central, semente, id_semente_bloco=9)
        print(f"[Centralizado] treino {t_c:.1f}s | melhor bloco {bloco_c} | PR-AUC val {hist_c['pr_auc_val'].max():.4f}")

        # (b) Federado + SMPC (modelo final)
        est_f, rodada_f, hist_f = executar_federado(dados, semente)
        t_f = hist_f['t_treino'].sum()

        # (c) Local isolado: cada cliente com as mesmas sementes que usa no federado
        avs_locais = []
        for k, (X_k, y_k) in enumerate(dados['clientes']):
            est_k, _, _, _ = treinar_em_blocos(dados, X_k, y_k, semente, id_semente_bloco=k)
            avs_locais.append(avaliar(est_k['D'], dados))
        print(f"[Local isolado] PR-AUC teste por cliente: {[round(a['PR-AUC'], 4) for a in avs_locais]}")

        av = {CENTRAL: avaliar(est_c['D'], dados), FEDERADO: avaliar(est_f['D'], dados),
              LOCAL: {m: np.mean([a[m] for a in avs_locais]) for m in METRICAS}}
        for cfg in CONFIGS:
            linhas.append({'Semente': semente, 'Configuração': cfg, **{m: av[cfg][m] for m in METRICAS}})
            print(f"  {cfg:16s} ROC-AUC {av[cfg]['ROC-AUC']:.4f} | PR-AUC {av[cfg]['PR-AUC']:.4f} | F1 {av[cfg]['F1']:.4f}")
        tempos.append({'Semente': semente, 'Centralizado': t_c, 'Federado (sequencial)': t_f,
                       'Federado (paralelo, estimado)': t_f / NUM_CLIENTES,
                       'SMPC total': hist_f['t_smpc'].sum(), 'Erro máx. SMPC': hist_f['erro_max'].max(),
                       'Bloco escolhido (central)': bloco_c, 'Rodada escolhida (federado)': rodada_f})
        pares_scores.append((av[CENTRAL]['scores'], av[FEDERADO]['scores']))
        curvas.append((hist_c.assign(semente=semente, cfg=CENTRAL), hist_f[['rodada', 'pr_auc_val']].assign(semente=semente,
        if exemplo is None:
            exemplo = {cfg: av[cfg]['scores'] for cfg in [CENTRAL, FEDERADO]}

    df = pd.DataFrame(linhas)
    df_tempos = pd.DataFrame(tempos).set_index('Semente')
    df_curvas = pd.concat([pd.concat(c) for c in curvas], ignore_index=True)

    # ----- Resumo -----
    resumo = df.groupby('Configuração')[METRICAS].agg(['mean', 'std']).reindex(CONFIGS)
    print(f"\n>> MÉTRICAS NO TESTE: média ± desvio-padrão em {len(SEMENTES)} sementes")
    print(pd.DataFrame({m: [f"{resumo.loc[c, (m, 'mean')]:.4f} ± {np.nan_to_num(resumo.loc[c, (m, 'std')]):.4f}"
                            for c in CONFIGS] for m in METRICAS}, index=CONFIGS).to_string())

    piv = df.pivot(index='Semente', columns='Configuração')
    deltas = {m: (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).values for m in METRICAS}
    print(f"\n>> Δ PAREADO POR SEMENTE ({FEDERADO} − {CENTRAL}):")
    for m in METRICAS:
        d = deltas[m]
        meia = stats.t.ppf(0.975, len(d) - 1) * d.std(ddof=1) / np.sqrt(len(d)) if len(d) > 1 else 0.0
        print(f"  {m:9s}: {d.mean():+.4f}  (IC95% entre sementes: [{d.mean() - meia:+.4f}, {d.mean() + meia:+.4f}])")

    boot = bootstrap_delta(dados['y_test'], pares_scores, N_BOOTSTRAP, SEMENTES[0])
    ic = {m: np.percentile(v, [2.5, 97.5]) for m, v in boot.items()}
    print(f"\n>> IC 95% POR BOOTSTRAP DO Δ MÉDIO ({N_BOOTSTRAP} reamostragens do teste):")
    for m in ['ROC-AUC', 'PR-AUC']:
        veredito = 'não-inferior' if ic[m][0] > -MARGEM_NAO_INFERIORIDADE else 'não-inferioridade NÃO demonstrada'
        print(f"  Δ{m}: médio {deltas[m].mean():+.4f} | IC95% [{ic[m][0]:+.4f}, {ic[m][1]:+.4f}] -> {veredito} "
              f"(margem {MARGEM_NAO_INFERIORIDADE})")
```

```
nao_inferior = ic[METRICA_PRINCIPAL][0] > -MARGEM_NAO_INFERIORIDADE
print(f"\n>> RESPOSTA À QUESTÃO DE PESQUISA (métrica principal: {METRICA_PRINCIPAL}):")
if nao_inferior:
    print(f"  O STEP-GAN Federado + SMPC é NÃO-INFERIOR ao baseline centralizado: limite inferior do IC95% "
          f"{ic[METRICA_PRINCIPAL][0]:+.4f} > {-MARGEM_NAO_INFERIORIDADE}.")
else:
    print(f"  NÃO foi possível demonstrar não-inferioridade: limite inferior do IC95% "
          f"{ic[METRICA_PRINCIPAL][0]:+.4f} <= {-MARGEM_NAO_INFERIORIDADE}.")
ganho = (piv[(METRICA_PRINCIPAL, FEDERADO)] - piv[(METRICA_PRINCIPAL, LOCAL)]).mean()
print(f"  Ganho da colaboração ({FEDERADO} − {LOCAL}): {ganho:+.4f} {METRICA_PRINCIPAL}")
print(f"  SMPC: erro máx. vs FedAvg em claro {df_tempos['Erro máx. SMPC'].max():.1e}")

print("\n>> TEMPOS (s) E CHECKPOINTS ESCOLHIDOS POR SEMENTE:")
print(df_tempos.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}"}, float_format=lambda x: f"{x:.2f}"))

arquivo = f'resultados_teste6_{nome_base}.csv'
df.to_csv(arquivo, index=False)
print(f"Resultados por semente salvos em '{arquivo}'.")
resultados_gerais[nome_base] = dict(metricas=df, tempos=df_tempos, ic_bootstrap=ic, nao_inferior=nao_inferior)

# ----- Visualizações -----
fig, axes = plt.subplots(2, 3, figsize=(22, 11))
fig.suptitle(f"Teste 6 (v2): Centralizado × Federado + SMPC (STEP-GAN v5, {nome_base}, {len(SEMENTES)} sementes)",
             fontsize=14, fontweight='bold')

# 1. Métricas
ax = axes[0, 0]
largura = 0.8 / len(CONFIGS)
x = np.arange(len(METRICAS))
for i, cfg in enumerate(CONFIGS):
    ax.bar(x + (i - (len(CONFIGS) - 1) / 2) * largura, [resumo.loc[cfg, (m, 'mean')] for m in METRICAS], largura,
           yerr=[np.nan_to_num(resumo.loc[cfg, (m, 'std')]) for m in METRICAS], capsize=3,
           color=CORES[cfg], label=cfg)
ax.set_xticks(x)
ax.set_xticklabels(METRICAS)
ax.set_ylim(0, 1.2)
ax.set_title('Métricas no teste (média ± desvio)', fontweight='bold')
ax.legend(loc='upper center', ncol=3)

# 2. Curvas ROC (primeira semente)
ax = axes[0, 1]
for cfg in [CENTRAL, FEDERADO]:
    fpr, tpr, _ = roc_curve(dados['y_test'], exemplo[cfg])
    ax.plot(fpr, tpr, '-' if cfg == CENTRAL else '--', color=CORES[cfg],
            label=f"{cfg} (AUC={roc_auc_score(dados['y_test'], exemplo[cfg]):.4f})")
ax.plot([0, 1], [0, 1], ':', color='gray')
ax.set_title(f'Curva ROC no teste (semente {SEMENTES[0]})', fontweight='bold')
ax.set_xlabel('FPR')
ax.set_ylabel('TPR')
ax.legend(loc='lower right')

# 3. Convergência: PR-AUC na validação por bloco/rodada
ax = axes[0, 2]
for cfg in [CENTRAL, FEDERADO]:
    sub = df_curvas[df_curvas['cfg'] == cfg]
    for _, g in sub.groupby('semente'):
        ax.plot(g['rodada'], g['pr_auc_val'], color=CORES[cfg], alpha=0.25)
    media = sub.groupby('rodada')['pr_auc_val'].mean()
    ax.plot(media.index, media.values, color=CORES[cfg], lw=2.5, marker='o', label=f'{cfg} (média)')
ax.set_xlabel(f'Rodada / bloco de {EPOCAS_LOCAIS} épocas')
ax.set_title('PR-AUC na validação', fontweight='bold')
ax.legend()

# 4. Δ por semente e IC de não-inferioridade
ax = axes[1, 0]
for i, m in enumerate(['ROC-AUC', 'PR-AUC', 'F1']):
    ax.scatter(np.full(len(deltas[m]), i), deltas[m], color=CORES[FEDERADO], zorder=3,
               label='Δ por semente' if i == 0 else None)
    ax.plot([i - 0.2, i + 0.2], [deltas[m].mean()] * 2, color='black', lw=2, label='Δ médio' if i == 0 else None)
for i, m in enumerate(['ROC-AUC', 'PR-AUC']):
    ax.errorbar(i + 0.3, np.mean(ic[m]), yerr=[[np.mean(ic[m]) - ic[m][0]], [ic[m][1] - np.mean(ic[m])]],
                fmt='none', ecolor='#2ca02c', capsize=6, lw=2, label='IC95% bootstrap' if i == 0 else None)
ax.axhline(0, color='gray', lw=1)
ax.axhline(-MARGEM_NAO_INFERIORIDADE, color='#d62728', ls='--',
           label=f'Margem de não-inferioridade (−{MARGEM_NAO_INFERIORIDADE})')
ax.set_xticks(range(3))
ax.set_xticklabels(['ΔROC-AUC', 'ΔPR-AUC', 'ΔF1'])
ax.set_title(f'{FEDERADO} − {CENTRAL}', fontweight='bold')
ax.legend(loc='best', fontsize=9)

# 5. Tempo de treino
ax = axes[1, 1]
media_t = df_tempos[['Centralizado', 'Federado (sequencial)', 'Federado (paralelo, estimado)']].mean()
```

```
    media_t = df_tempos[['Centralizado', 'Federado (Sequencial)', 'Federado (Paralelo, estimado)']].mean()
    ax.bar(media_t.index, media_t.values, color=['#1f77b4', '#ff9896', '#d62728'])
    for i, v in enumerate(media_t.values):
        ax.text(i, v, f"{v:.1f}s", ha='center', va='bottom')
    ax.set_title(f"Tempo de treino (SMPC total: {df_tempos['SMPC total'].mean():.2f}s)", fontweight='bold')
    ax.set_ylabel('segundos')

    axes[1, 2].axis('off')
    plt.tight_layout()
    plt.show()

print("\n[SUCESSO] Teste 6 (v2) concluído na base BAF!")
```

```
================================================================================
 BASE: BAF | 5 sementes | 5 x 7 épocas | 3 geradores
================================================================================
Carregando '/kaggle/input/bank-account-fraud-dataset-neurips-2022/Base.csv'...
[DADOS] Features: 51 | Treino: 600000 (fraudes: 6617) | Validação: 200000 (fraudes: 2206) | Teste: 200000 (fraudes: 2206)

--- Semente 42 ---
[Centralizado] treino 588.2s | melhor bloco 5 | PR-AUC val 0.1447
  Rodada 1/5 | PR-AUC val 0.1343 | treino 117.7s | SMPC 214 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1384 | treino 116.9s | SMPC 233 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1397 | treino 117.6s | SMPC 221 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1406 | treino 117.6s | SMPC 236 ms (erro máx. vs claro 8.8e-08)
  Rodada 5/5 | PR-AUC val 0.1420 | treino 116.6s | SMPC 215 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.1302), np.float64(0.1333), np.float64(0.1328)]
  Centralizado     ROC-AUC 0.8830 | PR-AUC 0.1367 | F1 0.0143
  Federado + SMPC  ROC-AUC 0.8820 | PR-AUC 0.1338 | F1 0.0196
  Local isolado    ROC-AUC 0.8794 | PR-AUC 0.1321 | F1 0.0155

--- Semente 43 ---
[Centralizado] treino 591.3s | melhor bloco 5 | PR-AUC val 0.1463
  Rodada 1/5 | PR-AUC val 0.1347 | treino 118.1s | SMPC 211 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1380 | treino 118.8s | SMPC 232 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1402 | treino 118.4s | SMPC 224 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1416 | treino 120.5s | SMPC 237 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.1424 | treino 119.4s | SMPC 218 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.1315), np.float64(0.1336), np.float64(0.1332)]
  Centralizado     ROC-AUC 0.8835 | PR-AUC 0.1383 | F1 0.0152
  Federado + SMPC  ROC-AUC 0.8817 | PR-AUC 0.1346 | F1 0.0152
  Local isolado    ROC-AUC 0.8793 | PR-AUC 0.1327 | F1 0.0176

--- Semente 44 ---
[Centralizado] treino 586.4s | melhor bloco 5 | PR-AUC val 0.1477
  Rodada 1/5 | PR-AUC val 0.1344 | treino 117.3s | SMPC 221 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1375 | treino 116.8s | SMPC 236 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1403 | treino 117.2s | SMPC 219 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1416 | treino 117.2s | SMPC 233 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.1428 | treino 117.3s | SMPC 221 ms (erro máx. vs claro 8.9e-08)
[Local isolado] PR-AUC teste por cliente: [np.float64(0.1312), np.float64(0.1338), np.float64(0.1336)]
  Centralizado     ROC-AUC 0.8843 | PR-AUC 0.1378 | F1 0.0248
  Federado + SMPC  ROC-AUC 0.8826 | PR-AUC 0.1354 | F1 0.0188
  Local isolado    ROC-AUC 0.8796 | PR-AUC 0.1329 | F1 0.0198

--- Semente 45 ---
[Centralizado] treino 592.7s | melhor bloco 4 | PR-AUC val 0.1454
  Rodada 1/5 | PR-AUC val 0.1356 | treino 118.1s | SMPC 211 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1373 | treino 117.2s | SMPC 223 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1394 | treino 117.7s | SMPC 213 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1401 | treino 117.8s | SMPC 234 ms (erro máx. vs claro 8.9e-08)
```

```
# ====================================================================
# --- 5d. EXPERIMENTO NA BASE INSURANCE CLAIMS (mesma configuração da seção 5) ---
# ====================================================================
from sklearn.metrics import roc_curve

METRICA_PRINCIPAL = 'ROC-AUC'       # Métrica do teste de não-inferioridade ('ROC-AUC' ou 'PR-AUC')
MARGEM_NAO_INFERIORIDADE = 0.01     # Perda máxima aceitável do federado em relação ao centralizado
N_BOOTSTRAP = 300                   # Reamostragens do teste para o IC 95% do Δ médio

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CENTRAL, FEDERADO, LOCAL = 'Centralizado', 'Federado + SMPC', 'Local isolado'
CONFIGS = [CENTRAL, FEDERADO, LOCAL]
CORES = {CENTRAL: '#1f77b4', FEDERADO: '#d62728', LOCAL: '#7f7f7f'}


def treinar_em_blocos(dados, X, y, semente, id_semente_bloco):
    """
    Treino SEM federação com o mesmo procedimento do FL: parte dos mesmos pesos iniciais, treina NUM_RODADAS blocos
    de EPOCAS_LOCAIS épocas e guarda o checkpoint com melhor PR-AUC na validação.
    """
    input_dim = dados['input_dim']
    seed_everything(semente)
    estado = novo_estado_global(input_dim)          # Mesma inicialização de executar_federado()
    historico, melhor, t_total = [], None, 0.0
    for bloco in range(1, NUM_RODADAS + 1):
        sincronizar()
        t0 = time.perf_counter()
        estado = treinar_local(estado, X, y, input_dim, semente * 1000 + bloco * 10 + id_semente_bloco)
        sincronizar()
        t_total += time.perf_counter() - t0
        pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado['D'], input_dim, dados['X_val']))
        historico.append(dict(rodada=bloco, pr_auc_val=pr_auc_val))
        if melhor is None or pr_auc_val > melhor[0]:
            melhor = (pr_auc_val, bloco, copy.deepcopy(estado))
    return melhor[2], melhor[1], pd.DataFrame(historico), t_total
```

```
def bootstrap_delta(y, pares_scores, n_boot, semente):
    """IC 95% do Δ médio entre sementes (federado − centralizado), reamostrando o conjunto de teste."""
    rng = np.random.default_rng(semente)
    deltas = {'ROC-AUC': [], 'PR-AUC': []}
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
        if y[idx].min() == y[idx].max():
            continue
        for m, fn in [('ROC-AUC', roc_auc_score), ('PR-AUC', average_precision_score)]:
            deltas[m].append(np.mean([fn(y[idx], f[idx]) - fn(y[idx], c[idx]) for c, f in pares_scores]))
    return {m: np.array(v) for m, v in deltas.items()}


resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_INSURANCE:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue
    X_central = np.concatenate([c[0] for c in dados['clientes']])   # União exata dos dados dos clientes
    y_central = np.concatenate([c[1] for c in dados['clientes']])

    linhas, tempos, pares_scores, curvas, exemplo = [], [], [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        # (a) Centralizado
        est_c, bloco_c, hist_c, t_c = treinar_em_blocos(dados, X_central, y_central, semente, id_semente_bloco=9)
        print(f"[Centralizado] treino {t_c:.1f}s | melhor bloco {bloco_c} | PR-AUC val {hist_c['pr_auc_val'].max():.4f}")

        # (b) Federado + SMPC (modelo final)
        est_f, rodada_f, hist_f = executar_federado(dados, semente)
        t_f = hist_f['t_treino'].sum()

        # (c) Local isolado: cada cliente com as mesmas sementes que usa no federado
        avs_locais = []
        for k, (X_k, y_k) in enumerate(dados['clientes']):
            est_k, _, _, _ = treinar_em_blocos(dados, X_k, y_k, semente, id_semente_bloco=k)
            avs_locais.append(avaliar(est_k['D'], dados))
        print(f"[Local isolado] PR-AUC teste por cliente: {[round(a['PR-AUC'], 4) for a in avs_locais]}")

        av = {CENTRAL: avaliar(est_c['D'], dados), FEDERADO: avaliar(est_f['D'], dados),
              LOCAL: {m: np.mean([a[m] for a in avs_locais]) for m in METRICAS}}
        for cfg in CONFIGS:
            linhas.append({'Semente': semente, 'Configuração': cfg, **{m: av[cfg][m] for m in METRICAS}})
            print(f"  {cfg:16s} ROC-AUC {av[cfg]['ROC-AUC']:.4f} | PR-AUC {av[cfg]['PR-AUC']:.4f} | F1 {av[cfg]['F1']:.4f}'
        tempos.append({'Semente': semente, 'Centralizado': t_c, 'Federado (sequencial)': t_f,
                       'Federado (paralelo, estimado)': t_f / NUM_CLIENTES,
                       'SMPC total': hist_f['t_smpc'].sum(), 'Erro máx. SMPC': hist_f['erro_max'].max(),
                       'Bloco escolhido (central)': bloco_c, 'Rodada escolhida (federado)': rodada_f})
        pares_scores.append((av[CENTRAL]['scores'], av[FEDERADO]['scores']))
        curvas.append((hist_c.assign(semente=semente, cfg=CENTRAL), hist_f[['rodada', 'pr_auc_val']].assign(semente=semente
        if exemplo is None:
            exemplo = {cfg: av[cfg]['scores'] for cfg in [CENTRAL, FEDERADO]}

    df = pd.DataFrame(linhas)
    df_tempos = pd.DataFrame(tempos).set_index('Semente')
    df_curvas = pd.concat([pd.concat(c) for c in curvas], ignore_index=True)

    # ----- Resumo -----
    resumo = df.groupby('Configuração')[METRICAS].agg(['mean', 'std']).reindex(CONFIGS)
    print(f"\n>> MÉTRICAS NO TESTE: média ± desvio-padrão em {len(SEMENTES)} sementes")
    print(pd.DataFrame({m: [f"{resumo.loc[c, (m, 'mean')]:.4f} ± {np.nan_to_num(resumo.loc[c, (m, 'std')]):.4f}"
                            for c in CONFIGS] for m in METRICAS}, index=CONFIGS).to_string())

    piv = df.pivot(index='Semente', columns='Configuração')
    deltas = {m: (piv[(m, FEDERADO)] - piv[(m, CENTRAL)]).values for m in METRICAS}
    print(f"\n>> Δ PAREADO POR SEMENTE ({FEDERADO} − {CENTRAL}):")
    for m in METRICAS:
        d = deltas[m]
        meia = stats.t.ppf(0.975, len(d) - 1) * d.std(ddof=1) / np.sqrt(len(d)) if len(d) > 1 else 0.0
        print(f"  {m:9s}: {d.mean():+.4f}  (IC95% entre sementes: [{d.mean() - meia:+.4f}, {d.mean() + meia:+.4f}])")

    boot = bootstrap_delta(dados['y_test'], pares_scores, N_BOOTSTRAP, SEMENTES[0])
    ic = {m: np.percentile(v, [2.5, 97.5]) for m, v in boot.items()}
    print(f"\n>> IC 95% POR BOOTSTRAP DO Δ MÉDIO ({N_BOOTSTRAP} reamostragens do teste):")
    for m in ['ROC-AUC', 'PR-AUC']:
        veredito = 'não-inferior' if ic[m][0] > -MARGEM_NAO_INFERIORIDADE else 'não-inferioridade NÃO demonstrada'
```